

Yes. One important correction first:

`apply()`, `compose()`, `andThen()`, and `identity()` are associated with Java's `java.util.function.Function<T,R>` functional interface, but they are **not all the same kind of method**.

- `apply()` → **abstract method** (the Single Abstract Method / SAM)
- `compose()` → **default method**
- `andThen()` → **default method**
- `identity()` → **static method**, not a default method

This distinction is very important for understanding how Function works with lambdas.

1. What is `Function<T,R>`?

`Function<T,R>` is a built-in functional interface introduced in Java 8.

It represents:

A function that accepts one argument of type T and produces one result of type R.

Its conceptual form is:

`T ---> Function ---> R`

For example:

```
Function<Integer, Integer> square = x -> x * x;
```

Here:

Input = Integer
Output = Integer

So:

```
5
↓
x -> x * x
↓
25
```

The interface is conceptually:

```
@FunctionalInterface
public interface Function<T, R> {

    R apply(T t);
```



```
R apply(T t);
```

It:

Takes an input T and returns an output R.

Example:

```
Function<Integer, Integer> square = x -> x * x;
```

```
Integer result = square.apply(5);
```

```
System.out.println(result);
```

Output:

25

Execution flow

```
square.apply(5)
```

```
  |  
  v  
x = 5  
  |  
  v  
x * x  
  |  
  v  
5 * 5  
  |  
  v  
25
```

The lambda:

```
x -> x * x
```

provides the implementation of:

```
apply()
```

Conceptually:

```
Function<Integer, Integer> square =  
    new Function<Integer, Integer>() {  
  
        @Override  
        public Integer apply(Integer x) {
```

```
        return x * x;
    }
};
```

So:

`square.apply(5);`

is effectively executing:

`apply(5);`

4. Why is `apply()` abstract?

Because Function doesn't know what operation you want.

For example:

```
Function<Integer, Integer> square =
    x -> x * x;
```

Another function:

```
Function<Integer, Integer> cube =
    x -> x * x * x;
```

Another:

```
Function<Integer, Integer> increment =
    x -> x + 1;
```

All three use the same Function interface:

```
Function<Integer,Integer>
```

but provide different implementations of:

`apply()`

Therefore:

`apply()`

is the customizable operation.

5. compose() — execute BEFORE

This is one of the most important methods.

Definition

compose() combines two functions so that:

The supplied function executes first, and the current function executes second.

Suppose:

```
Function<Integer, Integer> multiplyBy2 =  
    x -> x * 2;
```

```
Function<Integer, Integer> add3 =  
    x -> x + 3;
```

Now:

```
Function<Integer, Integer> composed =  
    multiplyBy2.compose(add3);
```

The execution order is:

```
Input  
|  
v  
add3  
|  
v  
multiplyBy2  
|  
v  
Output
```

For input 5:

```
5  
↓  
add3  
↓  
8  
↓  
multiplyBy2  
↓  
16
```

Therefore:

```
System.out.println(composed.apply(5));
```

Output:

16

6. Understanding compose() mathematically

Suppose:

$$f = x \rightarrow x * 2$$
$$g = x \rightarrow x + 3$$

Then:

$$f.compose(g)$$

means:

$$f(g(x))$$

Therefore:

$$g(5) = 5 + 3 = 8$$
$$f(8) = 8 * 2 = 16$$

So:

$$f.compose(g).apply(5)$$

produces:

16

Key rule

$$f.compose(g)$$

↓

g executes first
f executes second

Remember:

compose = before

7. compose() example with different types

This becomes even more useful when input and output types differ.

```
Function<String, Integer> stringLength =  
    str -> str.length();
```

```
Function<Integer, Double> multiplyBy2 =  
    number -> number * 2.0;
```

We cannot directly use:

```
multiplyBy2.compose(stringLength)
```

unless the types line up appropriately.

Here they do:

```
String  
↓  
stringLength  
↓  
Integer  
↓  
multiplyBy2  
↓  
Double
```

So:

```
Function<String, Double> resultFunction =  
    multiplyBy2.compose(stringLength);
```

Then:

```
System.out.println(resultFunction.apply("Java"));
```

Execution:

```
"Java"  
↓  
length()  
↓  
4  
↓  
4 * 2.0  
↓
```

8.0

Output:

8.0

8. andThen() — execute AFTER

andThen() is almost the opposite mental model of compose().

Definition

andThen() combines functions so that:

The current function executes first, and the supplied function executes second.

Example:

```
Function<Integer, Integer> multiplyBy2 =  
    x -> x * 2;
```

```
Function<Integer, Integer> add3 =  
    x -> x + 3;
```

Now:

```
Function<Integer, Integer> result =  
    multiplyBy2.andThen(add3);
```

Execution:

```
Input  
|  
v  
multiplyBy2  
|  
v  
add3  
|  
v  
Output
```

For:

```
result.apply(5)
```

we get:

5
↓
multiplyBy2
↓
10
↓
add3
↓
13

Output:

13

9. andThen() mathematically

If:

$f = x \rightarrow x * 2$
 $g = x \rightarrow x + 3$

then:

`f.andThen(g)`

means:

$g(f(x))$

Therefore:

$f(5) = 10$

$g(10) = 13$

So:

`f.andThen(g).apply(5)`

returns:

13

Key rule

`f.andThen(g)`

↓

f executes first
g executes second

Remember:

andThen = after

10. compose() vs andThen()

This is one of the most common Java 8 interview questions.

Consider:

```
Function<Integer, Integer> f =  
    x -> x * 2;
```

```
Function<Integer, Integer> g =  
    x -> x + 3;
```

compose()

f.compose(g)

means:

$g \rightarrow f$

Therefore:

f.compose(g).apply(5)

becomes:

```
5  
↓  
g  
↓  
8  
↓  
f  
↓  
16
```

Result:

16

andThen()

`f.andThen(g)`

means:

$f \rightarrow g$

Therefore:

`f.andThen(g).apply(5)`

becomes:

5
↓
f
↓
10
↓
g
↓
13

Result:

13

Comparison

Expression	Execution order	Mathematical form	Result for 5
f.compose(g)	$g \rightarrow f$	$f(g(5))$	16
f.andThen(g)	$f \rightarrow g$	$g(f(5))$	13

11. Easy way to remember them

Think about English:

compose()

`f.compose(g)`

Read it as:

"Compose f with g."

But operationally:

g BEFORE f

andThen()

f.andThen(g)

Read it as:

"Do f, and then do g."

Therefore:

f BEFORE g

The simplest memory trick is:

compose → BEFORE

andThen → AFTER

12. identity() — return the input unchanged

Now we come to identity().

There is an important distinction:

identity()

is **not a default method**.

It is a:

static method of the Function interface.

Its conceptual implementation is:

```
static <T> Function<T, T> identity() {  
    return t -> t;  
}
```

It returns a function that does nothing to its input.

Example:

```
Function<String, String> identity =  
    Function.identity();  
  
System.out.println(identity.apply("Java"));
```

Output:

Java

Because:

Input
↓
identity function
↓
same Input

13. Identity function

Mathematically:

$$f(x) = x$$

For example:

```
identity(5)    = 5  
identity("Java") = "Java"  
identity(10.5) = 10.5
```

Java:

```
Function<Integer, Integer> identity =  
    Function.identity();  
  
System.out.println(identity.apply(100));
```

Output:

100

14. Why is identity() static?

Because you don't need an existing Function object to create an identity function.

You write:

```
Function.identity()
```

not:

```
someFunction.identity()
```

It is a factory-like method that creates/returns an identity function.

For example:

```
Function<String, String> f = Function.identity();
```

Conceptually:

```
Function<String, String> f =  
    value -> value;
```

15. `identity()` is especially useful with `Map`

One classic example is `Collectors.groupingBy()`.

Suppose:

```
List<String> names =  
    Arrays.asList("A", "B", "C");
```

You could use:

```
Function<String, String> identity =  
    Function.identity();
```

Then:

```
Map<String, List<String>> map =  
    names.stream()  
        .collect(Collectors.groupingBy(  
            Function.identity()  
        ));
```

The identity function means:

Take each element

↓

Use the same element as the key

For:

A
B
C

the keys remain:

A
B
C

16. Complete example

Here is all four concepts together:

```
import java.util.function.Function;

public class FunctionDemo {

    public static void main(String[] args) {

        Function<Integer, Integer> multiplyBy2 =
            x -> x * 2;

        Function<Integer, Integer> add3 =
            x -> x + 3;

        // 1. apply()
        int result1 = multiplyBy2.apply(5);

        // 2. compose()
        int result2 =
            multiplyBy2
                .compose(add3)
                .apply(5);

        // 3. andThen()
        int result3 =
            multiplyBy2
                .andThen(add3)
                .apply(5);

        // 4. identity()
        Function<Integer, Integer> identity =
            Function.identity();

        int result4 = identity.apply(5);
```

```
        System.out.println(result1);
        System.out.println(result2);
        System.out.println(result3);
        System.out.println(result4);
    }
}
```

Output:

```
10
16
13
5
```

17. Detailed execution algorithm

Consider:

```
Function<Integer, Integer> f =
    x -> x * 2;
```

```
Function<Integer, Integer> g =
    x -> x + 3;
```

A. apply()

```
f.apply(5);
```

Algorithm:

1. Receive input 5
2. Pass 5 to lambda
3. Execute $x * 2$
4. Calculate $5 * 2$
5. Return 10

Result:

```
10
```

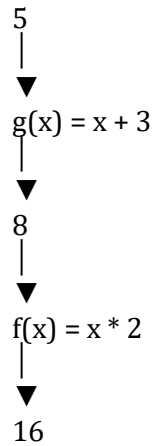
B. compose()

```
f.compose(g).apply(5);
```

Algorithm:

1. Create a composed Function
2. Receive input 5
3. Execute $g(5)$
4. $g(5) = 8$
5. Pass 8 to f
6. $f(8) = 16$
7. Return 16

Flow:



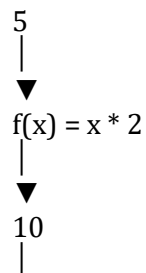
C. andThen()

`f.andThen(g).apply(5);`

Algorithm:

1. Create an andThen Function
2. Receive input 5
3. Execute $f(5)$
4. $f(5) = 10$
5. Pass 10 to g
6. $g(10) = 13$
7. Return 13

Flow:



▼
g(x) = x + 3
|
▼
13

D. identity()

Function.identity().apply(5);

Algorithm:

1. Call static identity()
2. Obtain identity Function
3. Pass 5 to the Function
4. Function returns the same 5
5. Return 5

Flow:

5
|
▼
identity
|
▼
5

18. Internal conceptual implementation

Understanding the conceptual implementation makes the whole topic much easier.

A simplified version of Function looks like:

```
@FunctionalInterface
public interface Function<T, R> {

    R apply(T t);

    default <V> Function<V, R>
        compose(Function<? super V, ? extends T> before) {

        return (V v) -> apply(before.apply(v));
    }

    default <V> Function<T, V>
        andThen(Function<? super R, ? extends V> after) {
```

```

    return (T t) -> after.apply(apply(t));
}

static <T> Function<T, T> identity() {
    return t -> t;
}
}

```

This simplified version reveals the entire logic.

compose()

```
return v -> apply(before.apply(v));
```

Therefore:

```

before
↓
current function

```

andThen()

```
return t -> after.apply(apply(t));
```

Therefore:

```

current function
↓
after

```

identity()

```
return t -> t;
```

Therefore:

```
input = output
```

19. Why are compose() and andThen() default methods?

Java 8 introduced default methods so interfaces could provide reusable implementations without forcing every implementing class to implement them.

For example:

```
Function<Integer, Integer> square =  
    x -> x * x;
```

You only provide:

```
apply()
```

You don't have to implement:

```
compose()  
andThen()
```

because they already have implementations in the Function interface.

So the lambda only supplies the **abstract method**.

20. Why doesn't a lambda implement compose()?

Consider:

```
Function<Integer, Integer> square =  
    x -> x * x;
```

A common misconception is:

"The lambda implements all methods of Function."

No.

It implements the **single abstract method**:

```
apply()
```

The other methods already come from the interface.

Conceptually:

Function	
├── apply()	← Lambda provides implementation
├── compose()	← Function interface provides default implementation
├── andThen()	← Function interface provides default implementation
└── identity()	← Function interface provides static implementation

21. Functional interface rule

Function<T,R> is a functional interface because it has exactly **one abstract method**:

```
R apply(T t);
```

It can have any number of:

- default methods
- static methods
- methods inherited from Object that don't count as abstract interface methods

For example:

```
@FunctionalInterface
interface MyFunction<T, R> {

    R apply(T value);

    default void method1() {
    }

    default void method2() {
    }

    static void method3() {
    }

    static void method4() {
    }
}
```

It can still be a functional interface because only:

```
apply()
```

is abstract.

22. Chaining multiple functions

The real power of compose() and andThen() is function chaining.

For example:

```
Function<String, String> trim =  
    s -> s.trim();
```

```
Function<String, String> upper =  
    s -> s.toUpperCase();
```

```
Function<String, Integer> length =  
    s -> s.length();
```

Now:

```
Function<String, Integer> process =  
    trim  
    .andThen(upper)  
    .andThen(length);
```

Input:

```
" java "
```

Execution:

```
" java "  
  ↓  
trim()  
  ↓  
"java"  
  ↓  
toUpperCase()  
  ↓  
"JAVA"  
  ↓  
length()  
  ↓  
4
```

Therefore:

```
System.out.println(process.apply(" java "));
```

Output:

```
4
```

This is an important example of the **functional programming style** in Java.

23. compose() chaining

Suppose:

```
Function<String, String> trim =  
    s -> s.trim();
```

```
Function<String, String> upper =  
    s -> s.toUpperCase();
```

```
Function<String, Integer> length =  
    s -> s.length();
```

You could write:

```
Function<String, Integer> process =  
    length  
    .compose(upper)  
    .compose(trim);
```

Execution:

```
Input  
↓  
trim  
↓  
upper  
↓  
length  
↓  
Output
```

For:

```
" java "
```

the result is:

4

Notice that compose() is read from the **outside toward the input** when reasoning about execution.

24. Important type-flow concept

Function composition requires compatible types.

Suppose:

Function<A, B>

Function<B, C>

These can be chained:

$A \rightarrow B \rightarrow C$

For example:

```
Function<String, Integer> f =  
    s -> s.length();
```

```
Function<Integer, Double> g =  
    i -> i * 2.5;
```

Then:

```
Function<String, Double> result =  
    g.compose(f);
```

Type flow:

```
String  
↓  
f  
↓  
Integer  
↓  
g  
↓  
Double
```

This is one of the fundamental ideas behind Function<T,R>.

25. Function<T,R> vs Consumer<T> vs Predicate<T> vs Supplier<T>

These four are frequently confused.

Functional interface	Input	Output	Main method
Function<T,R>	1	1	apply()
Consumer<T>	1	0	accept()

Predicate<T>	1	boolean	test()
Supplier<T>	0	1	get()

Examples:

```
Function<Integer, Integer> f =
    x -> x * 2;
Consumer<String> c =
    x -> System.out.println(x);
Predicate<Integer> p =
    x -> x > 10;
Supplier<Double> s =
    () -> Math.random();
```

Their conceptual flows are:

Function:

$T \rightarrow R$

Consumer:

$T \rightarrow \text{void}$

Predicate:

$T \rightarrow \text{boolean}$

Supplier:

$\text{nothing} \rightarrow T$

26. One very important correction about terminology

You wrote:

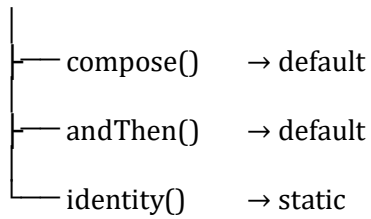
"Function functional interfaces include default (built-in) methods such as `apply()`, `compose()`, `andThen()`, `identity()`"

A more technically accurate statement is:

`java.util.function.Function<T,R>` is a functional interface whose abstract method is `apply()`, whose default methods include `compose()` and `andThen()`, and whose static method includes `identity()`.

So:

```
Function<T,R>
|
└─ apply()      → abstract
```



This distinction is especially important in **Java 8 interviews**, because the interviewer may ask:

"How many abstract methods does Function have?"

Answer:

Exactly one: `apply()`

Or:

"Is `identity()` a default method?"

Answer:

No.

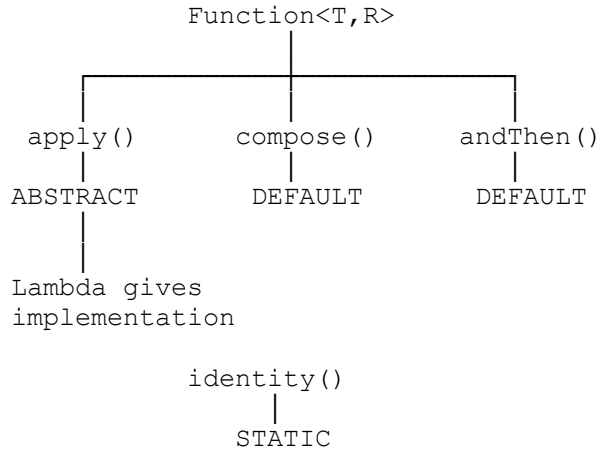
`identity()` is a static method.

27. Final mental model

Remember the following four statements:

```
apply()
↓
EXECUTE the function
compose()
↓
DO BEFORE
andThen()
↓
DO AFTER
identity()
↓
RETURN SAME INPUT
```

And the complete structure:



The easiest interview mnemonic is:

apply = execute
compose = before
andThen = after
identity = same value

So, for:

```
Function<Integer, Integer> f = x -> x * 2;  
Function<Integer, Integer> g = x -> x + 3;
```

you should immediately think:

```
f.apply(5)  
→ 10
```

```
f.compose(g).apply(5)  
→ g(5) → f(8) → 16
```

```
f.andThen(g).apply(5)  
→ f(5) → g(10) → 13
```

```
Function.identity().apply(5)  
→ 5
```

That is the core execution model of Java 8's `Function<T,R>`.

Yes. One important correction first: in Java 8, Predicate<T> has **one abstract method** test(), but and(), or(), and negate() are **default methods**, while isEqual() is a **static method**.

1. What is Predicate<T>?

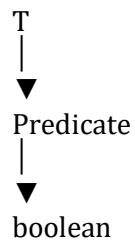
Predicate<T> is a built-in functional interface in:

```
java.util.function.Predicate
```

It represents a condition that:

Accepts one input and returns true or false.

Conceptually:



For example:

```
Predicate<Integer> isEven = n -> n % 2 == 0;
```

```
System.out.println(isEven.test(10));  
System.out.println(isEven.test(11));
```

Output:

```
true  
false
```

The lambda:

```
n -> n % 2 == 0
```

provides the implementation of the functional interface's abstract method:

```
boolean test(T t);
```

2. Structure of Predicate<T>

The important methods can be summarized as:

Method	Method type	Purpose
test()	Abstract	Tests the condition
and()	Default	Combines predicates using logical AND
or()	Default	Combines predicates using logical OR
negate()	Default	Reverses the result
isEqual()	Static	Creates a predicate that tests equality

So the structure is:

```
Predicate<T>
├── test()    → abstract
├── and()     → default
├── or()      → default
├── negate()  → default
└── isEqual() → static
```

This is an important distinction.

Your statement:

"Predicate functional interfaces include default methods such as test(), and(), or(), negate(), isEqual()."

should technically be corrected to:

Predicate<T> is a functional interface whose abstract method is test(), whose default methods include and(), or(), and negate(), and whose static method includes isEqual().

3. test() — the main Predicate operation

Definition

The abstract method is:

```
boolean test(T t);
```

It:

Accepts an object of type T and returns a boolean value.

Example:

```
Predicate<Integer> isPositive =  
    n -> n > 0;
```

```
System.out.println(isPositive.test(10));  
System.out.println(isPositive.test(-5));
```

Output:

```
true  
false
```

Execution:

```
10  
↓  
n > 0  
↓  
true
```

and:

```
-5  
↓  
n > 0  
↓  
false
```

4. Why is test() abstract?

Because the Predicate interface cannot know what condition you want.

For example:

```
Predicate<Integer> isEven =  
    n -> n % 2 == 0;
```

Another:

```
Predicate<Integer> isGreaterThan100 =
```

```
n -> n > 100;
```

Another:

```
Predicate<String> isEmpty =  
    s -> s.isEmpty();
```

All of them use:

```
Predicate<T>
```

but have different implementations of:

```
test()
```

Conceptually:

```
Predicate<Integer> isEven =  
    new Predicate<Integer>() {  
  
        @Override  
        public boolean test(Integer n) {  
            return n % 2 == 0;  
        }  
    };
```

The lambda is therefore providing the implementation of test().

5. and() — logical AND

and() is a **default method**.

Definition

and() combines two predicates so that:

Both predicates must return true.

Suppose:

```
Predicate<Integer> greaterThan10 =  
    n -> n > 10;
```

```
Predicate<Integer> lessThan100 =  
    n -> n < 100;
```

Combine them:

```
Predicate<Integer> between10And100 =  
    greaterThan10.and(lessThan100);
```

Now:

```
System.out.println(between10And100.test(50));
```

Output:

true

Because:

```
50 > 10 → true  
50 < 100 → true
```

```
true AND true  
↓  
true
```

For 150:

```
150 > 10 → true  
150 < 100 → false
```

```
true AND false  
↓  
false
```

6. and() truth table

and() follows normal Boolean AND logic:

Predicate A	Predicate B	A AND B
true	true	true
true	false	false
false	true	false
false	false	false

Conceptually:

```
p1.and(p2)
```

means:

```
x -> p1.test(x) && p2.test(x)
```

So:

```
Predicate<Integer> p =  
    greaterThan10.and(lessThan100);
```

can be understood conceptually as:

```
Predicate<Integer> p =  
    x -> greaterThan10.test(x)  
        && lessThan100.test(x);
```

7. or() — logical OR

or() is also a **default method**.

Definition

or() combines two predicates so that:

At least one predicate must return true.

Example:

```
Predicate<Integer> lessThan10 =  
    n -> n < 10;
```

```
Predicate<Integer> greaterThan100 =  
    n -> n > 100;
```

```
Predicate<Integer> outsideRange =  
    lessThan10.or(greaterThan100);
```

Now:

```
System.out.println(outsideRange.test(5));
```

Output:

true

because:

5 < 10 → true
5 > 100 → false

true OR false
↓
true

For 50:

50 < 10 → false
50 > 100 → false

false OR false
↓
false

8. or() truth table

Predicate A	Predicate B	A OR B
true	true	true
true	false	true
false	true	true
false	false	false

Conceptually:

p1.or(p2)

means:

x -> p1.test(x) || p2.test(x)

9. negate() — reverse the result

negate() is a **default method**.

Definition

negate() creates a new predicate whose result is the opposite of the original predicate.

Suppose:

```
Predicate<Integer> positive =  
    n -> n > 0;
```

Then:

```
Predicate<Integer> notPositive =  
    positive.negate();
```

Now:

```
System.out.println(positive.test(10));  
System.out.println(notPositive.test(10));
```

Output:

```
true  
false
```

Because:

```
positive.test(10)  
  ↓  
  true  
  ↓  
  negate()  
  ↓  
  false
```

10. negate() mathematically

If:

$P(x)$

is the original predicate, then:

$\text{negate}(P(x)) = \text{NOT } P(x)$

For example:

```
Predicate<Integer> even =  
    n -> n % 2 == 0;
```

```
Predicate<Integer> odd =  
    even.negate();
```

Then:

```
System.out.println(odd.test(5));
```

Output:

```
true
```

Conceptually:

```
even.negate()
```

is approximately:

```
x -> !even.test(x)
```

11. isEqual() — equality predicate

isEqual() is different from the previous three.

It is a:

static method

not a default method.

Its purpose is to create a predicate that checks whether its input is equal to a specified target value.

Example:

```
Predicate<String> isJava =  
    Predicate.isEqual("Java");
```

Then:

```
System.out.println(isJava.test("Java"));  
System.out.println(isJava.test("Python"));
```

Output:

```
true  
false
```

Conceptually:

```
Predicate.isEqual("Java")
```

creates something similar to:

```
value -> Objects.equals("Java", value)
```

This is particularly useful when you want to turn an equality comparison into a Predicate.

12. isEqual() with numbers

```
Predicate<Integer> is100 =  
    Predicate.isEqual(100);
```

```
System.out.println(is100.test(100));  
System.out.println(is100.test(200));
```

Output:

```
true  
false
```

Conceptually:

```
Input  
↓  
Is input equal to 100?  
↓  
true / false
```

13. Why is isEqual() static?

You don't need an existing predicate object to create the equality predicate.

You call:

```
Predicate.isEqual("Java");
```

rather than:

```
somePredicate.isEqual("Java");
```

This is similar to a factory method.

It creates a new predicate based on the supplied target value.

14. Combining and(), or(), and negate()

This is where Predicate becomes especially powerful.

Suppose we have:

```
Predicate<Integer> greaterThan18 =  
    age -> age > 18;
```

```
Predicate<Integer> lessThan60 =  
    age -> age < 60;
```

We can combine them:

```
Predicate<Integer> workingAge =  
    greaterThan18.and(lessThan60);
```

Now:

```
System.out.println(workingAge.test(30));
```

Output:

true

The logical expression is:

age > 18 AND age < 60

15. Complex predicate

Suppose we want:

Age must be greater than 18 AND less than 60, OR age must equal 65.

We can write:

```
Predicate<Integer> greaterThan18 =  
    age -> age > 18;
```

```
Predicate<Integer> lessThan60 =  
    age -> age < 60;
```

```
Predicate<Integer> is65 =  
    Predicate.isEqual(65);
```

```
Predicate<Integer> eligible =
```

```
greaterThan18
    .and(lessThan60)
    .or(is65);
```

Conceptually:

(age > 18 AND age < 60) OR age == 65

Test:

```
System.out.println(eligible.test(30));
System.out.println(eligible.test(10));
System.out.println(eligible.test(65));
```

Output:

```
true
false
true
```

16. Important: Predicate composition is left-to-right

Consider:

```
p1.and(p2).or(p3)
```

The logical structure is effectively:

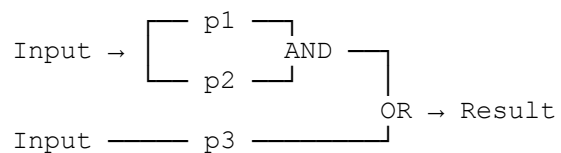
(p1 AND p2) OR p3

For example:

```
Predicate<Integer> p1 = n -> n > 10;
Predicate<Integer> p2 = n -> n < 100;
Predicate<Integer> p3 = n -> n == 500;
```

```
Predicate<Integer> result =
    p1.and(p2).or(p3);
```

Conceptually:



17. Short-circuit behavior

`and()` and `or()` use short-circuit Boolean logic.

This is very important.

`and()`

For:

`p1.and(p2)`

if `p1` returns false, Java does not need to evaluate `p2`.

Example:

```
Predicate<Integer> p1 =  
    n -> n > 100;
```

```
Predicate<Integer> p2 =  
    n -> {  
        System.out.println("p2 executed");  
        return n < 200;  
    };
```

```
Predicate<Integer> combined =  
    p1.and(p2);
```

```
System.out.println(combined.test(50));
```

Output:

false

"p2 executed" is not printed because:

`p1` → false

and:

false AND anything = false

18. `or()` short-circuit behavior

For:

`p1.or(p2)`

if p1 returns true, Java does not need to evaluate p2.

Example:

```
Predicate<Integer> p1 =  
    n -> n > 10;
```

```
Predicate<Integer> p2 =  
    n -> {  
        System.out.println("p2 executed");  
        return n < 100;  
    };
```

```
Predicate<Integer> combined =  
    p1.or(p2);
```

```
System.out.println(combined.test(50));
```

Output:

true

p2 does not need to execute because:

true OR anything = true

19. Complete example using all methods

```
import java.util.function.Predicate;
```

```
public class PredicateDemo {
```

```
    public static void main(String[] args) {
```

```
        // test()  
        Predicate<Integer> isEven =  
            n -> n % 2 == 0;
```

```
        System.out.println(  
            isEven.test(10)  
        );
```

```
        // and()  
        Predicate<Integer> greaterThan10 =  
            n -> n > 10;
```

```
        Predicate<Integer> lessThan100 =  
            n -> n < 100;
```

```

    Predicate<Integer> between10And100 =
        greaterThan10.and(lessThan100);

    System.out.println(
        between10And100.test(50)
    );

    // or()
    Predicate<Integer> lessThan10 =
        n -> n < 10;

    Predicate<Integer> greaterThan100 =
        n -> n > 100;

    Predicate<Integer> outsideRange =
        lessThan10.or(greaterThan100);

    System.out.println(
        outsideRange.test(150)
    );

    // negate()
    Predicate<Integer> notEven =
        isEven.negate();

    System.out.println(
        notEven.test(11)
    );

    // isEqual()
    Predicate<Integer> is100 =
        Predicate.isEqual(100);

    System.out.println(
        is100.test(100)
    );
}
}

```

Output:

```

true
true
true
true
true

```

20. Conceptual implementation

A simplified version of Predicate<T> helps reveal how everything works:

```
@FunctionalInterface
public interface Predicate<T> {

    boolean test(T t);

    default Predicate<T> and(Predicate<? super T> other) {
        return t -> test(t) && other.test(t);
    }

    default Predicate<T> or(Predicate<? super T> other) {
        return t -> test(t) || other.test(t);
    }

    default Predicate<T> negate() {
        return t -> !test(t);
    }

    static <T> Predicate<T> isEqual(Object targetRef) {
        return targetRef == null
            ? Objects::isNull
            : object -> targetRef.equals(object);
    }
}
```

This simplified implementation is extremely useful for understanding the internal logic.

21. How test() works internally

Suppose:

```
Predicate<Integer> even =
    n -> n % 2 == 0;
```

When you execute:

```
even.test(10);
```

the conceptual flow is:

1. Input = 10
↓
2. Invoke Predicate.test(10)
↓

3. Lambda receives $n = 10$

↓

4. Evaluate $n \% 2 == 0$

↓

5. Evaluate $10 \% 2 == 0$

↓

6. Evaluate $0 == 0$

↓

7. Return true

22. How and() works internally

Suppose:

```
Predicate<Integer> p1 =  
    n -> n > 10;
```

```
Predicate<Integer> p2 =  
    n -> n < 100;
```

Then:

```
Predicate<Integer> p3 =  
    p1.and(p2);
```

Conceptually, `and()` creates another predicate:

```
n -> p1.test(n) && p2.test(n)
```

So:

```
p3.test(50)
```

becomes:

```
p1.test(50)
```

↓

```
true
```

↓

```
p2.test(50)
```

↓

```
true
```

↓

```
true
```

23. How or() works internally

```
Predicate<Integer> p3 =  
    p1.or(p2);
```

conceptually creates:

```
n -> p1.test(n) || p2.test(n)
```

Therefore:

```
p1.test(n)  
  ↓  
 true?  
 /  \  
yes  no  
 ↓   ↓  
true p2.test(n)
```

This is why or() is short-circuiting.

24. How negate() works internally

Given:

```
Predicate<Integer> positive =  
    n -> n > 0;
```

Then:

```
Predicate<Integer> negativeOrZero =  
    positive.negate();
```

is conceptually:

```
n -> !positive.test(n)
```

For:

```
negativeOrZero.test(-10)
```

the flow is:

```
positive.test(-10)  
  ↓  
 false
```

↓
!
↓
true

25. Predicate with collections

One of the most common real-world uses is filtering collections.

Example:

```
List<Integer> numbers =  
    Arrays.asList(10, 15, 20, 25, 30);
```

Create a predicate:

```
Predicate<Integer> isEven =  
    n -> n % 2 == 0;
```

Use it with Stream:

```
numbers.stream()  
    .filter(isEven)  
    .forEach(System.out::println);
```

Output:

```
10  
20  
30
```

The important relationship is:

```
Predicate  
↓  
filter()  
↓  
test()
```

Conceptually, `filter()` asks:

For each element:
 `predicate.test(element)`

If it returns:

`true` → keep element

false → discard element

26. Multiple predicates with Stream API

Suppose:

```
List<Integer> numbers =  
    Arrays.asList(5, 10, 15, 20, 25, 30, 40);
```

We want:

Numbers greater than 10 AND less than 30.

Create:

```
Predicate<Integer> greaterThan10 =  
    n -> n > 10;
```

```
Predicate<Integer> lessThan30 =  
    n -> n < 30;
```

Combine:

```
Predicate<Integer> condition =  
    greaterThan10.and(lessThan30);
```

Then:

```
numbers.stream()  
    .filter(condition)  
    .forEach(System.out::println);
```

Output:

```
15  
20  
25
```

The algorithm is:

10 → 10 > 10? → false → discard

15 → true AND true → keep

20 → true AND true → keep

25 → true AND true → keep

30 → true AND false → discard

40 → true AND false → discard

27. Predicate vs Function

This distinction is extremely important.

Function<T,R>

Transforms a value:

$T \rightarrow R$

Example:

```
Function<String, Integer> length =  
    s -> s.length();  
"Java" → 4
```

Predicate<T>

Tests a value:

$T \rightarrow \text{boolean}$

Example:

```
Predicate<String> isLong =  
    s -> s.length() > 5;  
"Java" → false  
"Programming" → true
```

Therefore:

Function
↓
transform

Predicate
↓
test / validate / filter

28. Predicate vs Consumer vs Supplier

Interface	Input	Output	Main method	Typical purpose
Function<T,R>	1	1	apply()	Transform
Predicate<T>	1	boolean	test()	Test/filter
Consumer<T>	1	0	accept()	Consume/use
Supplier<T>	0	1	get()	Supply/create

Examples:

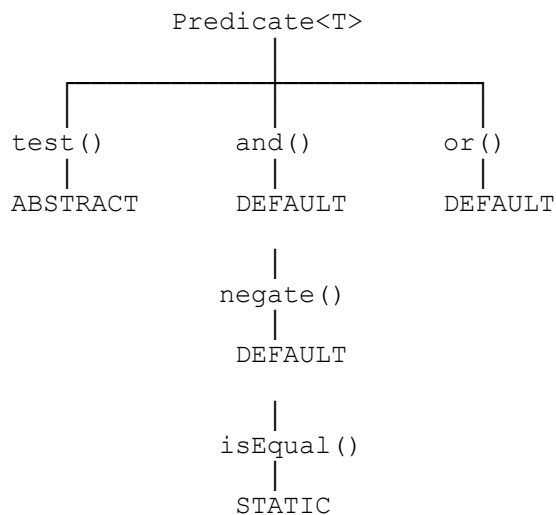
```
Function<Integer, Integer> square =  
    x -> x * x;  
Predicate<Integer> even =  
    x -> x % 2 == 0;  
Consumer<Integer> print =  
    x -> System.out.println(x);  
Supplier<Integer> random =  
    () -> 100;
```

Think:

Function → "What value should I produce?"
Predicate → "Is this condition true?"
Consumer → "What should I do with this value?"
Supplier → "What value should I provide?"

29. Functional-interface classification

For Predicate<T>:



Therefore, the **one and only abstract method** is:

```
boolean test(T t);
```

This is why this is legal:

```
Predicate<Integer> p =  
    x -> x > 10;
```

The lambda implements test().

30. Interview-ready summary

If an interviewer asks:

What is Predicate<T>?

Predicate<T> is a Java 8 functional interface representing a boolean-valued condition. It accepts one argument of type T and returns true or false.

What is its abstract method?

```
boolean test(T t);
```

What are its important default methods?

```
and()  
or()  
negate()
```

What is its important static method?

```
isEqual()
```

What does and() do?

```
p1.and(p2)
```

means:

p1 AND p2

What does or() do?

`p1.or(p2)`

means:

`p1 OR p2`

What does `negate()` do?

`p.negate()`

means:

`NOT p`

What does `isEqual()` do?

`Predicate.isEqual(value)`

creates a predicate that tests whether the input is equal to the specified value.

Final mental model

`Predicate<T>`

`T → boolean`

`test()`



execute/test condition

`and()`



condition1 AND condition2

`or()`



condition1 OR condition2

`negate()`



reverse true ↔ false

`isEqual()`



create equality predicate

The easiest mnemonic is:

test() = TEST
and() = BOTH
or() = EITHER
negate() = OPPOSITE
isEqual() = EQUALITY

And the key Java 8 distinction is:

test() → abstract
and() → default
or() → default
negate() → default
isEqual() → static

That distinction is what makes Predicate<T> a **functional interface** even though it contains several other methods.

Yes. One important correction first: in Java 8, `Consumer<T>` has **one abstract method**, `accept()`. The `andThen()` method is a **default method**.

So the technically correct statement is:

`Consumer<T>` is a functional interface whose abstract method is `accept(T)`, and whose default method `andThen(Consumer<? super T>)` allows two consumers to be chained sequentially.

1. What is `Consumer<T>`?

`Consumer<T>` is a built-in functional interface in:

`java.util.function.Consumer`

It represents an operation that:

Accepts one input argument and performs some action without returning a result.

Its basic form is:

$T \rightarrow \text{void}$

Compare this with `Function`:

`Function`:

$T \rightarrow R$

`Consumer`:

$T \rightarrow \text{void}$

For example:

```
Consumer<String> print =  
    name -> System.out.println(name);
```

Here:

Input = `String`

Output = nothing

Usage:

```
print.accept("Java");
```

Output:

2. Structure of Consumer<T>

The important structure is:

```
Consumer<T>
├── accept()      → abstract
└── andThen()    → default
```

Conceptually:

```
@FunctionalInterface
public interface Consumer<T> {

    void accept(T t);

    default Consumer<T> andThen(
        Consumer<? super T> after) {
        ...
    }
}
```

Therefore:

Method	Type	Purpose
accept()	Abstract	Performs the operation
andThen()	Default	Executes another Consumer after this one

There is **no static method such as identity()** in Consumer.

3. accept() — the main Consumer operation

Definition

The abstract method is:

```
void accept(T t);
```

It:

Receives an argument and performs an action, but returns no value.

Example:

```
Consumer<String> consumer =  
    name -> System.out.println(name);  
  
consumer.accept("Anurag");
```

Output:

Anurag

The execution is:

```
"Anurag"  
↓  
accept()  
↓  
System.out.println()  
↓  
display text  
↓  
no return value
```

4. Why is accept() abstract?

Because the Consumer interface doesn't know what action you want to perform.

For example:

Print something

```
Consumer<String> print =  
    s -> System.out.println(s);
```

Convert to uppercase and print

```
Consumer<String> upperPrint =  
    s -> System.out.println(s.toUpperCase());
```

Save something

```
Consumer<Employee> save =  
    employee -> employeeRepository.save(employee);
```

Send an email

```
Consumer<String> sendEmail =  
    email -> sendEmailToUser(email);
```

All of these are different operations.

Therefore, Java provides the abstract method:

```
void accept(T t);
```

and the lambda supplies its implementation.

5. Lambda expression and accept()

Consider:

```
Consumer<Integer> printSquare =  
    x -> System.out.println(x * x);
```

The lambda:

```
x -> System.out.println(x * x)
```

provides the implementation of:

```
accept(Integer x)
```

Conceptually, it is similar to:

```
Consumer<Integer> printSquare =  
    new Consumer<Integer>() {  
  
        @Override  
        public void accept(Integer x) {  
            System.out.println(x * x);  
        }  
    };
```

Then:

```
printSquare.accept(5);
```

produces:

25

6. Consumer does not return anything

This is the most important characteristic of Consumer.

For example:

```
Consumer<Integer> square =  
    x -> System.out.println(x * x);
```

This is valid:

```
square.accept(5);
```

But this is not:

```
int result = square.accept(5);
```

because:

```
accept()
```

returns:

void

If you need a returned value, use `Function<T,R>` instead.

For example:

```
Function<Integer, Integer> square =  
    x -> x * x;
```

```
int result = square.apply(5);
```

Result:

25

7. andThen() — chaining Consumers

`andThen()` is the important default method of Consumer.

Definition

`andThen()`:

Creates a combined Consumer that executes the current Consumer first and another Consumer afterward.

Suppose:

```
Consumer<String> first =  
    s -> System.out.println("First: " + s);
```

```
Consumer<String> second =  
    s -> System.out.println("Second: " + s);
```

Combine them:

```
Consumer<String> combined =  
    first.andThen(second);
```

Now:

```
combined.accept("Java");
```

Output:

```
First: Java  
Second: Java
```

Execution order:

```
"Java"  
↓  
first.accept()  
↓  
First: Java  
↓  
second.accept()  
↓  
Second: Java
```

8. andThen() means "do this, then that"

The easiest way to remember:

```
first.andThen(second)
```

means:

first → second

or:

1. Execute first
2. Execute second

It does **not** mean:

second → first

For example:

```
Consumer<Integer> c1 =  
    x -> System.out.println("C1: " + x);
```

```
Consumer<Integer> c2 =  
    x -> System.out.println("C2: " + x);
```

```
Consumer<Integer> c3 =  
    c1.andThen(c2);
```

```
c3.accept(100);
```

Output:

```
C1: 100  
C2: 100
```

9. Conceptual implementation of andThen()

The default implementation can be understood approximately as:

```
default Consumer<T> andThen(  
    Consumer<? super T> after) {  
  
    return t -> {  
        accept(t);  
        after.accept(t);  
    };  
}
```

This is the key to understanding it.

Suppose:

```
Consumer<String> c1 =  
    s -> System.out.println("First");
```

```
Consumer<String> c2 =
```

```
s -> System.out.println("Second");
```

Then:

```
Consumer<String> c3 =  
    c1.andThen(c2);
```

is conceptually similar to:

```
Consumer<String> c3 =  
    s -> {  
        c1.accept(s);  
        c2.accept(s);  
    };
```

Therefore:

```
c3.accept("Java");
```

executes:

```
c1.accept("Java")  
  ↓  
c2.accept("Java")
```

10. Multiple `andThen()` calls

You can chain more than two Consumers.

For example:

```
Consumer<String> c1 =  
    s -> System.out.println("Step 1");
```

```
Consumer<String> c2 =  
    s -> System.out.println("Step 2");
```

```
Consumer<String> c3 =  
    s -> System.out.println("Step 3");
```

```
Consumer<String> process =  
    c1.andThen(c2).andThen(c3);
```

```
process.accept("Java");
```

Output:

Step 1

Step 2

Step 3

Execution:

Input

↓

c1

↓

c2

↓

c3

↓

End

This is called **Consumer chaining**.

11. Real-world example: Employee processing

Suppose we have:

```
class Employee {  
  
    private String name;  
    private double salary;  
  
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
}
```

Now create Consumers:

```
Consumer<Employee> printName =  
    e -> System.out.println(  
        "Name: " + e.getName()  
    );
```

```
Consumer<Employee> printSalary =  
    e -> System.out.println(  
        "Salary: " + e.getSalary()  
    );
```

Combine:

```
Consumer<Employee> displayEmployee =  
    printName.andThen(printSalary);
```

Execute:

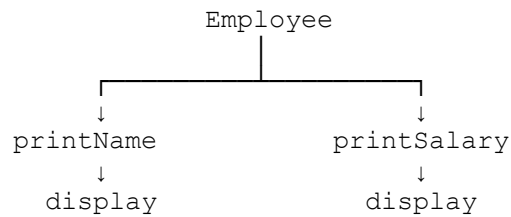
```
Employee emp =  
    new Employee("Rahul", 75000);
```

```
displayEmployee.accept(emp);
```

Output:

```
Name: Rahul  
Salary: 75000.0
```

The important thing is that **the Employee is passed to both Consumers**.



12. Consumer with forEach()

One of the most common uses of Consumer is:

forEach()

For example:

```
List<String> names =  
    Arrays.asList(  
        "Amit",  
        "Rahul",  
        "Anurag"  
    );
```

```
names.forEach(  
    e -> System.out.println(e);
```

```
    name -> System.out.println(name)
};
```

The lambda:

```
name -> System.out.println(name)
```

is a `Consumer<String>`.

Conceptually:

```
Consumer<String> consumer =
    name -> System.out.println(name);

names.forEach(consumer);
```

So:

```
List
↓
forEach()
↓
Consumer.accept()
↓
process each element
```

13. `forEach()` execution algorithm

Suppose:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30);
```

and:

```
Consumer<Integer> print =
    n -> System.out.println(n);
```

Then:

```
numbers.forEach(print);
```

Conceptually:

```
1. Take 10
   ↓
   print.accept(10)
```

↓
print 10

2. Take 20
↓
print.accept(20)
↓
print 20

3. Take 30
↓
print.accept(30)
↓
print 30

Output:

10
20
30

14. Consumer vs Function

This distinction is very important.

Function

```
Function<Integer, Integer> doubleValue =  
    x -> x * 2;
```

Flow:

5
↓
Function
↓
10

It **returns a result**.

You call:

```
doubleValue.apply(5);
```

Consumer

```
Consumer<Integer> printDouble =
```

```
x -> System.out.println(x * 2);
```

Flow:

```
5
↓
Consumer
↓
print 10
↓
no result
```

You call:

```
printDouble.accept(5);
```

So:

Function → apply() → returns value

Consumer → accept() → returns nothing

15. Consumer vs Predicate

These are also easy to confuse.

Predicate

```
Predicate<Integer> isEven =
    x -> x % 2 == 0;
```

Returns:

true / false

Call:

```
isEven.test(10);
```

Consumer

```
Consumer<Integer> print =
    x -> System.out.println(x);
```

Returns:

nothing

Call:

```
print.accept(10);
```

Therefore:

Predicate:
 $T \rightarrow \text{boolean}$

Consumer:
 $T \rightarrow \text{void}$

16. Consumer vs Supplier

Consumer

```
Consumer<String> print =  
    s -> System.out.println(s);
```

Input:

String

Output:

nothing

Supplier

```
Supplier<String> supply =  
    () -> "Java";
```

Input:

nothing

Output:

String

So:

Consumer:
 $T \rightarrow \text{void}$

Supplier:
 $\text{void} \rightarrow T$

Conceptually:

Java Functional Interfaces

Function : $T \rightarrow R$

Predicate : $T \rightarrow \text{boolean}$

Consumer : $T \rightarrow \text{void}$

Supplier : $\text{void} \rightarrow T$

17. andThen() versus Function.andThen()

Both Function and Consumer have an andThen() method, but they serve slightly different purposes.

Function

```
Function<Integer, Integer> f =  
    x -> x * 2;
```

```
Function<Integer, Integer> g =  
    x -> x + 3;
```

```
Function<Integer, Integer> result =  
    f.andThen(g);
```

Here:

```
5  
↓  
f  
↓  
10  
↓  
g  
↓  
13
```

The output of f becomes the input of g.

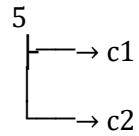
Consumer

```
Consumer<Integer> c1 =  
    x -> System.out.println("First: " + x);
```

```
Consumer<Integer> c2 =  
    x -> System.out.println("Second: " + x);
```

```
Consumer<Integer> result =  
    c1.andThen(c2);
```

Here:



There is **no output from c1 that becomes input to c2**.

Both Consumers receive the same original input.

This distinction is very important.

18. Consumer andThen() execution model

Consider:

```
Consumer<Integer> c1 =  
    x -> System.out.println(x * 2);
```

```
Consumer<Integer> c2 =  
    x -> System.out.println(x + 3);
```

```
Consumer<Integer> combined =  
    c1.andThen(c2);
```

Calling:

```
combined.accept(5);
```

does **not** mean:

```
c1:  
5 → 10
```

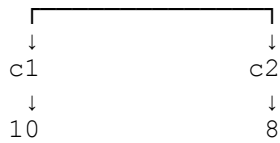
```
then c2:  
10 → 13
```

That would be a Function-style transformation.

Instead:

Instead:

Original input = 5



Output:

10
8

Because both Consumers receive 5.

19. Exception behavior of andThen()

Another important point is execution order.

Suppose:

```
Consumer<String> first =  
    s -> {  
        System.out.println("First");  
        throw new RuntimeException();  
    };
```

```
Consumer<String> second =  
    s -> System.out.println("Second");
```

```
Consumer<String> combined =  
    first.andThen(second);
```

Now:

```
combined.accept("Java");
```

Execution:

```
first  
↓  
exception  
↓  
second is NOT executed
```

Therefore:

First
Exception

and not:

First
Second

This happens because `andThen()` executes the second Consumer only after the first Consumer completes normally.

20. `andThen()` null behavior

The after Consumer must not be null.

For example:

```
Consumer<String> c1 =  
    s -> System.out.println(s);
```

```
Consumer<String> c2 = null;
```

```
Consumer<String> combined =  
    c1.andThen(c2);
```

This results in a `NullPointerException`.

The API requires the after Consumer to be non-null.

21. Complete example

```
import java.util.function.Consumer;
```

```
public class ConsumerDemo {
```

```
    public static void main(String[] args) {
```

```
        Consumer<String> first =  
            name -> System.out.println(  
                "Hello " + name  
            );
```

```
        Consumer<String> second =  
            name -> System.out.println(  
                "Welcome to Java"
```

```

        );

        Consumer<String> combined =
            first.andThen(second);

        combined.accept("Anurag");
    }
}

```

Output:

```

Hello Anurag
Welcome to Java

```

Execution:

```

"Anurag"
↓
first.accept()
↓
Hello Anurag
↓
second.accept()
↓
Welcome to Java

```

22. Anonymous class equivalent

The lambda:

```

Consumer<String> print =
    s -> System.out.println(s);

```

is conceptually similar to:

```

Consumer<String> print =
    new Consumer<String>() {

        @Override
        public void accept(String s) {
            System.out.println(s);
        }
    };

```

Then:

```

print.accept("Java");

```

calls the implemented:

`accept()`

method.

23. Functional-interface rule

A common question is:

If `Consumer` has `accept()` and `andThen()`, why is it still a functional interface?

Because:

`accept()`

is the only **abstract** method.

While:

`andThen()`

is already implemented as a **default method**.

Therefore:

```
Consumer<T>
├── accept() → abstract
└── andThen() → default
```

Only one abstract method is required for a functional interface.

Therefore this is legal:

```
Consumer<String> c =
    s -> System.out.println(s);
```

The lambda implements only:

`accept()`

24. Complete conceptual implementation

A simplified version of Consumer can be understood as:

```
@FunctionalInterface
public interface Consumer<T> {

    void accept(T t);

    default Consumer<T> andThen(
        Consumer<? super T> after) {

        Objects.requireNonNull(after);

        return (T t) -> {
            accept(t);
            after.accept(t);
        };
    }
}
```

This reveals the complete logic.

accept()

```
void accept(T t);
```

Means:

```
Receive input
↓
Perform action
↓
Return nothing
```

andThen()

```
return t -> {
    accept(t);
    after.accept(t);
};
```

Means:

```
Receive input
↓
Current Consumer
↓
After Consumer
```

↓
Return nothing

25. Multiple Consumers in a business workflow

Imagine an employee-processing workflow:

```
Consumer<Employee> validate =  
    e -> System.out.println("Validating employee");
```

```
Consumer<Employee> save =  
    e -> System.out.println("Saving employee");
```

```
Consumer<Employee> notify =  
    e -> System.out.println("Sending notification");
```

Chain them:

```
Consumer<Employee> workflow =  
    validate  
        .andThen(save)  
        .andThen(notify);
```

Execute:

```
workflow.accept(employee);
```

Algorithm:

```
Employee  
↓  
validate  
↓  
save  
↓  
notify  
↓  
complete
```

Output:

```
Validating employee  
Saving employee  
Sending notification
```

This is a good real-world example of why `Consumer.andThen()` exists.

26. Consumer with Stream API

Consider:

```
List<String> names =  
    Arrays.asList(  
        "Amit",  
        "Rahul",  
        "Anurag"  
    );
```

You can create:

```
Consumer<String> printUpperCase =  
    name -> System.out.println(  
        name.toUpperCase()  
    );
```

Then:

```
names.forEach(printUpperCase);
```

Output:

```
AMIT  
RAHUL  
ANURAG
```

The relationship is:

```
Stream/Collection  
    ↓  
    forEach()  
    ↓  
    Consumer.accept()  
    ↓  
    perform action
```

27. Combining Predicate and Consumer

This is a very common Java 8 pattern.

Suppose:

```
Predicate<Integer> isEven =  
    n -> n % 2 == 0;
```

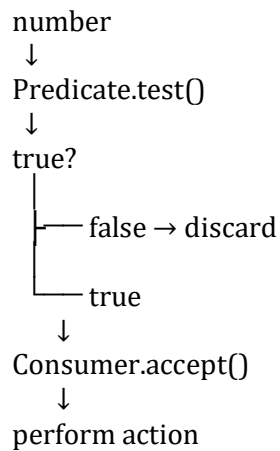
```
Consumer<Integer> print =
```

```
n -> System.out.println(  
    "Even: " + n  
);
```

Then:

```
numbers.stream()  
    .filter(isEven)  
    .forEach(print);
```

The execution becomes:



For:

10, 11, 12, 13

the result is:

Even: 10
Even: 12

This demonstrates the different responsibilities:

Predicate → decides
Consumer → acts

28. Consumer and forEach()

This is worth remembering for interviews.

When you write:

```
list.forEach(  
    item -> System.out.println(item)
```

);

the lambda is effectively a:

Consumer<T>

because forEach() needs something that:

takes one element

↓

performs an action

↓

returns nothing

For example:

```
Consumer<String> consumer =  
    item -> System.out.println(item);
```

```
list.forEach(consumer);
```

29. Comparison with the other Java 8 functional interfaces

Interface	Abstract method	Input	Output	Purpose
Function<T,R>	apply()	1	1	Transform
Predicate<T>	test()	1	boolean	Test
Consumer<T>	accept()	1	0	Perform action
Supplier<T>	get()	0	1	Supply value

A useful mental model is:

Function

↓

"Give me a value, I'll transform it."

Predicate

↓

"Give me a value, I'll tell you true/false."

Consumer

↓

"Give me a value, I'll do something with it."

Supplier



"Don't give me anything, I'll give you a value."

30. Final interview-ready summary

What is Consumer<T>?

Consumer<T> is a Java 8 functional interface that accepts one argument and performs an operation without returning a result.

Its functional shape is:

$T \rightarrow \text{void}$

What is its abstract method?

`void accept(T t);`

What is its default method?

`Consumer<T> andThen(Consumer<? super T> after);`

What does accept() do?

It executes the Consumer's action.

`consumer.accept(value);`

What does andThen() do?

It creates a Consumer that executes:

current Consumer



after Consumer

For example:

`c1.andThen(c2)`

means:

`c1.accept(value)`



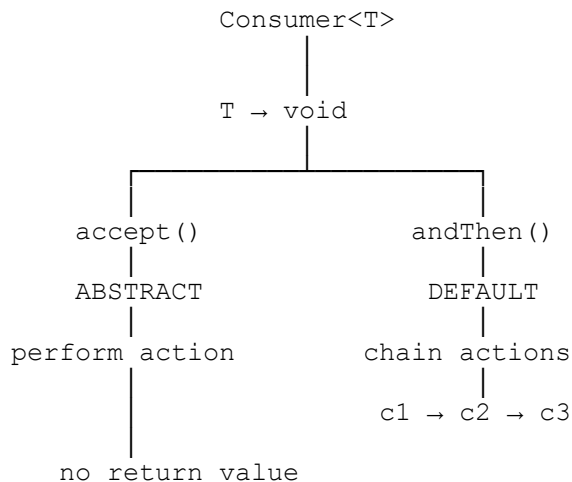
`c2.accept(value)`

Does c2 receive the output of c1?

No.

Both receive the **same original input** because Consumer returns void.

31. Final mental model



The easiest mnemonic is:

`accept()` = DO SOMETHING

`andThen()` = DO SOMETHING ELSE AFTER THAT

And the critical distinction from Function is:

Function:

input → transformation → output

Consumer:

input → action → nothing

So when you see:

`Consumer<T>`

think:

"I have a value, and I want to perform an action on it—not calculate and return another value."

Yes. One important correction first:

Supplier<T> is a functional interface whose abstract method is get(). get() is not a default method. In fact, Supplier<T> does not define a default method like Consumer.andThen() or Function.compose().

The technically correct structure is:

```
Supplier<T>  
└─ get() → abstract
```

1. What is Supplier<T>?

Supplier<T> is a built-in functional interface in:

java.util.function.Supplier

It represents:

A provider of a result, where no input argument is required.

Its functional shape is:

void → T

or:

no input → output

For example:

```
Supplier<String> supplier =  
    () -> "Java";
```

Calling:

```
String value = supplier.get();
```

produces:

Java

The basic flow is:

```
No input  
↓  
Supplier
```

↓
get()
↓
T

2. Structure of Supplier<T>

The interface is essentially:

```
@FunctionalInterface
public interface Supplier<T> {

    T get();
}
```

Therefore:

Method	Type	Purpose
get()	Abstract	Supplies/returns a value

Unlike the interfaces we discussed earlier:

```
Function<T,R>
    apply()      → abstract
    compose()   → default
    andThen()    → default
    identity()   → static
```

```
Predicate<T>
    test()       → abstract
    and()        → default
    or()         → default
    negate()     → default
    isEqual()    → static
```

```
Consumer<T>
    accept()     → abstract
    andThen()    → default
```

```
Supplier<T>
    get()        → abstract
```

So Supplier<T> is particularly simple.

3. What does get() mean?

get() means:

"Give me a value."

It accepts **no arguments**:

T get();

and returns a value of type T.

For example:

```
Supplier<Integer> supplier =  
    () -> 100;
```

```
Integer value = supplier.get();
```

```
System.out.println(value);
```

Output:

100

The flow is:

```
get()  
↓  
execute lambda  
↓  
return 100
```

4. Why is get() abstract?

Because Java does not know what value you want the Supplier to produce.

For example:

```
Supplier<String> name =  
    () -> "Anurag";
```

Another:

```
Supplier<Integer> number =  
    () -> 100;
```

Another:

```
Supplier<Double> price =  
    () -> 99.99;
```

Another:

```
Supplier<Date> date =  
    () -> new Date();
```

The interface only defines:

```
T get();
```

The lambda provides the actual implementation.

5. Lambda implementation of Supplier

Consider:

```
Supplier<String> supplier =  
    () -> "Hello Java";
```

The lambda:

```
() -> "Hello Java"
```

provides the implementation of:

```
String get();
```

Conceptually, this is similar to:

```
Supplier<String> supplier =  
    new Supplier<String>() {  
  
        @Override  
        public String get() {  
            return "Hello Java";  
        }  
    };
```

Then:

```
supplier.get();
```

executes the implementation.

6. Important difference: Supplier takes no argument

Compare:

Function

```
Function<Integer, Integer> square =  
    x -> x * x;
```

```
square.apply(5);
```

Input:

5

Output:

25

Supplier

```
Supplier<Integer> number =  
    () -> 25;
```

```
number.get();
```

Input:

nothing

Output:

25

Therefore:

Function:

$T \rightarrow R$

Supplier:

$\text{void} \rightarrow T$

7. Basic example

```
import java.util.function.Supplier;

public class SupplierDemo {

    public static void main(String[] args) {

        Supplier<String> supplier =
            () -> "Hello Java";

        String value = supplier.get();

        System.out.println(value);
    }
}
```

Output:

Hello Java

Execution:

```
supplier.get()
  ↓
execute lambda
  ↓
"Hello Java"
  ↓
return String
```

8. Supplier of Integer

```
Supplier<Integer> numberSupplier =
    () -> 100;

System.out.println(numberSupplier.get());
```

Output:

100

No argument is supplied:

```
numberSupplier.get()
```

The Supplier itself knows how to obtain the value.

9. Supplier of Double

```
Supplier<Double> priceSupplier =  
    () -> 999.99;
```

```
System.out.println(priceSupplier.get());
```

Output:

999.99

Again:

No input

↓

Supplier

↓

999.99

10. Supplier can calculate a value

A Supplier does not have to return a constant.

For example:

```
Supplier<Integer> randomNumber =  
    () -> (int) (Math.random() * 100);
```

Every time:

```
System.out.println(randomNumber.get());
```

is called, the lambda executes.

For example:

57

23

81

4

The exact values vary.

This demonstrates an important characteristic:

A Supplier can calculate or obtain a value when get() is called.

11. Supplier can create objects

Suppose:

```
class Employee {  
  
    String name;  
  
    Employee(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public String toString() {  
        return name;  
    }  
}
```

Create a Supplier:

```
Supplier<Employee> employeeSupplier =  
    () -> new Employee("Anurag");
```

Then:

```
Employee emp =  
    employeeSupplier.get();
```

```
System.out.println(emp);
```

Output:

Anurag

The Supplier is therefore useful when object creation should be deferred until needed.

12. Supplier and lazy execution

This is one of the most important concepts.

Consider:

```
Supplier<String> supplier =  
    () -> {  
        System.out.println("Supplier executed");  
        return "Java";  
    };
```

At this point:

```
Supplier<String> supplier = ...
```

the lambda body has not executed.

Nothing is printed.

Only when:

```
supplier.get();
```

is called does the lambda execute.

Output:

Supplier executed

then:

Java

So:

```
Create Supplier  
↓  
No execution yet  
↓  
get()  
↓  
Execute operation  
↓  
Return value
```

This is often described as **deferred/lazy execution**.

13. Supplier vs direct method call

Consider:

```
String value = getName();
```

The method executes immediately.

With:

```
Supplier<String> supplier =  
    () -> getName();
```

the `getName()` operation is represented as a function-like object and executes when:

```
supplier.get();
```

is called.

Therefore:

Direct call:

```
getName()
```

↓

execute now

Supplier:

```
() -> getName()
```

↓

store behavior

↓

```
get()
```

↓

execute later

This is one of the major reasons Suppliers are useful.

14. Supplier with a method reference

Supplier works very naturally with method references.

Suppose:

```
public static String getMessage() {  
    return "Hello Java";  
}
```

Instead of:

```
Supplier<String> supplier =
```

```
() -> getMessage();
```

you can write:

```
Supplier<String> supplier =  
    SupplierDemo::getMessage;
```

Then:

```
System.out.println(supplier.get());
```

Output:

Hello Java

The method reference:

SupplierDemo::getMessage

represents:

No argument

↓

getMessage()

↓

String

15. Supplier with constructor reference

This is another important Java 8 use.

Suppose:

```
class Employee {  
  
    Employee() {  
        System.out.println("Employee created");  
    }  
}
```

You can write:

```
Supplier<Employee> supplier =  
    Employee::new;
```

Then:

```
Employee emp =  
    supplier.get();
```

The constructor executes when `get()` is called.

Execution:

```
Employee::new  
  ↓  
Supplier  
  ↓  
get()  
  ↓  
new Employee()  
  ↓  
Employee object
```

This is an excellent example of the relationship between:

- functional interfaces
- lambda expressions
- method references
- constructor references.

16. Supplier with `Stream.generate()`

One of the most important uses of `Supplier` is:

`Stream.generate()`

For example:

```
Stream.generate(() -> Math.random())  
    .limit(5)  
    .forEach(System.out::println);
```

The argument:

`() -> Math.random()`

is a `Supplier`-like operation.

Conceptually:

```
Stream.generate()  
  ↓
```

Supplier.get()
↓
generate value
↓
Supplier.get()
↓
generate value
↓
...

Each invocation can generate another value.

17. Stream.generate() with explicit Supplier

You can make the Supplier explicit:

```
Supplier<Integer> supplier =  
    () -> (int) (Math.random() * 100);
```

```
Stream.generate(supplier)  
    .limit(5)  
    .forEach(System.out::println);
```

Possible output:

```
27  
81  
14  
65  
42
```

The Supplier provides a new value whenever the stream requests one.

18. Supplier with Optional.orElseGet()

Another important real-world use is:

Optional.orElseGet()

For example:

```
Optional<String> optional =  
    Optional.of("Java");
```

String result =

```
optional.orElseGet(  
    () -> "Default"  
);
```

Since the Optional already contains "Java":

```
result = "Java"
```

The Supplier provides the fallback value only when needed.

Compare:

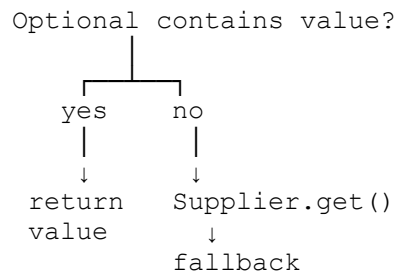
```
optional.orElse("Default");
```

with:

```
optional.orElseGet(() -> "Default");
```

orElseGet() accepts a Supplier.

Conceptually:



This is another example of deferred execution.

19. Supplier in Optional

Example:

```
Optional<String> name =  
    Optional.empty();
```

```
String result =  
    name.orElseGet(  
        () -> "Unknown"  
    );
```

```
System.out.println(result);
```

Output:

Unknown

Here:

() -> "Unknown"

acts as the Supplier of the fallback value.

20. Supplier vs Consumer

These are opposites in an important sense.

Consumer

input → action → nothing

Example:

```
Consumer<String> print =  
    s -> System.out.println(s);
```

Supplier

nothing → value

Example:

```
Supplier<String> supply =  
    () -> "Java";
```

Think:

Consumer:

"Give me something; I'll do something with it."

Supplier:

"Give me nothing; I'll give you something."

21. Supplier vs Function

Function

Function<String, Integer> length =

```
s -> s.length();
```

Requires input:

```
length.apply("Java");
```

Flow:

String

↓

Function

↓

Integer

Supplier

```
Supplier<Integer> number =  
    () -> 100;
```

Requires no input:

```
number.get();
```

Flow:

nothing

↓

Supplier

↓

Integer

Therefore:

```
Function<T,R>  
T → R
```

```
Supplier<T>  
void → T
```

22. Supplier vs Predicate

Predicate

```
Predicate<Integer> positive =  
    n -> n > 0;
```

Requires input:

positive.test(10);

Returns:

true / false

Supplier

```
Supplier<Integer> number =  
    () -> 10;
```

Requires no input:

```
number.get();
```

Returns:

10

Therefore:

Predicate:
 $T \rightarrow \text{boolean}$

Supplier:
 $\text{void} \rightarrow T$

23. Supplier vs Consumer vs Function vs Predicate

This is an excellent interview table:

Functional Interface	Input	Output	Method	Meaning
Function<T,R>	1	1	apply()	Transform
Predicate<T>	1	boolean	test()	Test
Consumer<T>	1	0	accept()	Perform action
Supplier<T>	0	1	get()	Provide value

Visual model:

Function

$T \longrightarrow R$

Predicate

$T \longrightarrow \text{boolean}$

Consumer

$T \longrightarrow \text{void}$

Supplier

$\text{void} \longrightarrow T$

This four-interface model is fundamental to Java 8 functional programming.

24. Why is Supplier a functional interface?

Because it has exactly one abstract method:

`T get();`

Therefore this is valid:

```
Supplier<String> supplier =  
    () -> "Java";
```

The lambda supplies the implementation of `get()`.

Conceptually:

```
Supplier<String> supplier =  
    new Supplier<String>() {  
  
        @Override  
        public String get() {  
            return "Java";  
        }  
    };
```

25. Is `get()` a default method?

No.

This is an important correction to your statement.

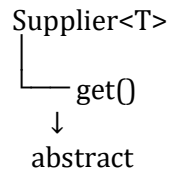
get() is:

abstract

not:

default

The correct structure is:



Because get() is the single abstract method, Supplier<T> qualifies as a functional interface.

26. Does Supplier have andThen()?

No.

This is another important distinction.

Compare:

Function	
└─ apply()	abstract
└─ compose()	default
└─ andThen()	default
└─ identity()	static
Predicate	
└─ test()	abstract
└─ and()	default
└─ or()	default
└─ negate()	default
└─ isEqual()	static
Consumer	
└─ accept()	abstract
└─ andThen()	default
Supplier	
└─ get()	abstract

Supplier is intentionally very small.

27. Complete Supplier example

```
import java.util.function.Supplier;

public class SupplierDemo {

    public static void main(String[] args) {

        Supplier<String> messageSupplier =
            () -> "Hello Java";

        Supplier<Integer> numberSupplier =
            () -> 100;

        Supplier<Double> priceSupplier =
            () -> 999.99;

        System.out.println(
            messageSupplier.get()
        );

        System.out.println(
            numberSupplier.get()
        );

        System.out.println(
            priceSupplier.get()
        );
    }
}
```

Output:

```
Hello Java
100
999.99
```

28. Supplier with a method reference

```
import java.util.function.Supplier;

public class SupplierMethodReference {

    public static String getMessage() {
        return "Hello Java";
    }

    public static void main(String[] args) {
```

```

    Supplier<String> supplier =
        SupplierMethodReference::getMessage;

    System.out.println(
        supplier.get()
    );
}
}

```

Output:

Hello Java

The method reference:

SupplierMethodReference::getMessage

matches:

String get();

because:

```

get()
↓
getMessage()
↓
String

```

29. Supplier with constructor reference

```
import java.util.function.Supplier;
```

```
class Employee {
```

```

    Employee() {
        System.out.println("Employee constructor executed");
    }
}

```

```
public class SupplierConstructor {
```

```
    public static void main(String[] args) {
```

```

        Supplier<Employee> supplier =
            Employee::new;
    }
}

```

```

        System.out.println("Before get()");

        Employee employee =
            supplier.get();

        System.out.println("After get()");
    }
}

```

Output:

```

Before get()
Employee constructor executed
After get()

```

Notice that the constructor is not invoked merely by creating:

```

Supplier<Employee> supplier =
    Employee::new;

```

It is invoked when:

```
supplier.get();
```

executes.

30. Supplier and lazy execution

This example makes the idea very clear:

```

Supplier<String> supplier =
    () -> {
        System.out.println("Supplier is executing");
        return "Java";
    };

```

```
System.out.println("Supplier created");
```

```
String result =
    supplier.get();
```

```
System.out.println(result);
```

Output:

```

Supplier created
Supplier is executing

```

Java

Execution order:

1. Create Supplier
↓
2. Supplier body does NOT execute
↓
3. Call get()
↓
4. Lambda body executes
↓
5. Return "Java"

This is why Suppliers are useful for **deferred value creation**.

31. A practical configuration example

Suppose a value may be expensive to calculate.

```
Supplier<String> configuration =  
    () -> {  
        System.out.println(  
            "Loading configuration..."  
        );  
  
        return "Production Configuration";  
    };
```

Nothing happens yet.

Later:

```
String config =  
    configuration.get();
```

Now the configuration is loaded.

This allows the code that needs the value to control **when the computation occurs**.

32. Supplier with a database-style example

Conceptually:

```
Supplier<List<Employee>> employees =  
    () -> employeeRepository.findAll();
```

Then:

```
List<Employee> list =  
    employees.get();
```

The Supplier represents:

"When you need employees, execute this operation and give me the result."

This is useful when an API expects a deferred computation.

33. Supplier with Optional.orElseGet()

A practical example:

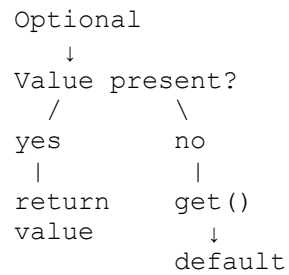
```
Optional<String> username =  
    Optional.empty();  
  
String name =  
    username.orElseGet(  
        () -> {  
            System.out.println(  
                "Generating default name"  
            );  
            return "Guest";  
        }  
    );  
  
System.out.println(name);
```

Output:

```
Generating default name  
Guest
```

The Supplier is invoked because the Optional is empty.

Conceptually:



34. Supplier and Stream.generate()

Another important Java 8 example:

```
Supplier<Integer> supplier =  
    () -> 10;
```

```
Stream.generate(supplier)  
    .limit(5)  
    .forEach(System.out::println);
```

Output:

```
10  
10  
10  
10  
10
```

Why?

Because Stream.generate() repeatedly asks the Supplier for a value:

```
Supplier.get()  
↓  
10
```

```
Supplier.get()  
↓  
10
```

```
Supplier.get()  
↓  
10
```

...

The limit(5) prevents infinite generation.

35. Supplier with random values

```
Supplier<Integer> randomNumber =  
    () -> (int) (Math.random() * 100);  
  
Stream.generate(randomNumber)  
    .limit(5)  
    .forEach(System.out::println);
```

Possible output:

```
42  
17  
89  
4  
63
```

Every call to:

```
randomNumber.get()
```

can produce a different value.

36. Supplier and method references

Supplier is also important for understanding method-reference syntax.

Suppose:

```
public static String createMessage() {  
    return "Hello";  
}
```

Lambda:

```
Supplier<String> supplier =  
    () -> createMessage();
```

Equivalent method reference:

```
Supplier<String> supplier =  
    SupplierDemo::createMessage;
```

The mapping is:

`() -> createMessage()`



`SupplierDemo::createMessage`

Both mean:

no arguments



`createMessage()`



`String`

37. Supplier and new

Constructor references work similarly.

Lambda:

```
Supplier<Employee> supplier =  
    () -> new Employee();
```

Equivalent constructor reference:

```
Supplier<Employee> supplier =  
    Employee::new;
```

Both represent:

no arguments



`new Employee()`



`Employee`

This is one of the most common examples of a constructor reference being assigned to a functional interface.

38. Internal execution algorithm

Consider:

```
Supplier<Integer> supplier =  
    () -> 100;
```

When:

```
supplier.get();
```

is executed:

Step 1:

Call supplier.get()

Step 2:

Dispatch to the Supplier implementation

Step 3:

Execute lambda body

Step 4:

Evaluate 100

Step 5:

Return Integer value 100

Flow:

```
supplier.get()
```

↓

```
lambda implementation
```

↓

```
return 100
```

There is no input parameter.

39. Supplier compared with a normal method

Normal method:

```
public static String getMessage() {  
    return "Java";  
}
```

Call:

```
String message =  
    getMessage();
```

Supplier:

```
Supplier<String> supplier =  
    () -> "Java";
```

Call:

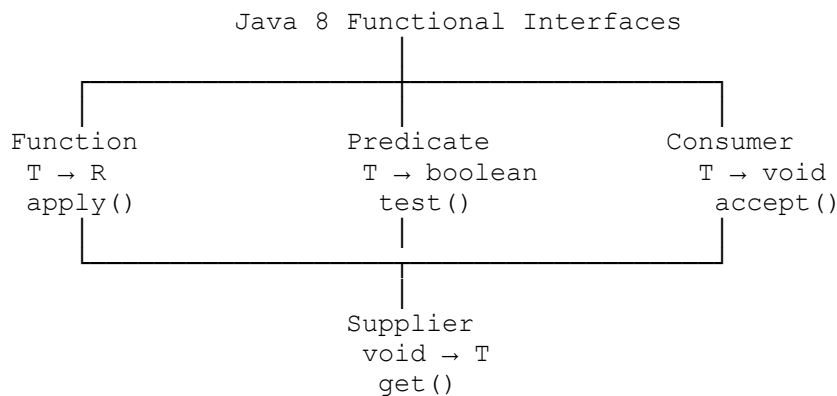
```
String message =  
    supplier.get();
```

The difference is that the Supplier allows you to treat the **behavior of obtaining the value as an object/function-like value**.

This is especially useful when another API wants a computation rather than an immediately computed value.

40. Final functional-interface hierarchy

You can remember the four primary Java 8 functional interfaces like this:



More simply:

Function	= INPUT → OUTPUT
Predicate	= INPUT → TRUE/FALSE
Consumer	= INPUT → ACTION
Supplier	= NO INPUT → OUTPUT

41. Interview-ready answer

If an interviewer asks:

What is Supplier<T>?

Supplier<T> is a Java 8 functional interface that represents a provider of a value. It takes no arguments and returns a value of type T.

What is its abstract method?

`T get();`

Is `get()` a default method?

No. It is the **single abstract method** of `Supplier<T>`.

Does `Supplier` have `andThen()`?

No. `Supplier<T>` does not define `andThen()`.

Why can a lambda implement `Supplier`?

Because `Supplier` has exactly one abstract method:

`T get();`

Therefore:

```
Supplier<String> s =  
    () -> "Java";
```

is valid.

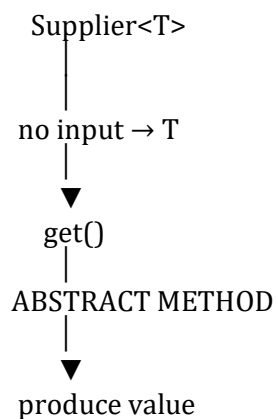
What is the functional shape?

$() \rightarrow T$

or:

no input $\rightarrow$ result

42. Final mental model



Example:

```
Supplier<String> supplier =  
    () -> "Java";
```

```
String value =  
    supplier.get();
```

Execution:

```
get()  
↓  
lambda  
↓  
"Java"  
↓  
return String
```

The easiest mnemonic is:

Supplier = "SUPPLY me a value."

get() = "GET the supplied value."

And the key correction to your original statement is:

```
Supplier<T>  
    get() → ABSTRACT
```

not:

```
get() → DEFAULT
```

Unlike Function, Predicate, and Consumer, the basic Supplier<T> interface has **only its single abstract get() method**; it does not need additional built-in composition methods to remain a functional interface.

Yes. One important correction first:

UnaryOperator<T> is a functional interface whose abstract method is apply(T). identity() is a static method, not a default method.

Also, UnaryOperator<T> inherits the default andThen() and compose() methods from Function<T,T>.

So the technically correct structure is:

```
UnaryOperator<T>
├── apply(T)      → abstract/inherited from Function
├── compose()     → default/inherited from Function
├── andThen()     → default/inherited from Function
└── identity()   → static
```

1. What is UnaryOperator<T>?

UnaryOperator<T> is a built-in functional interface in:

java.util.function.UnaryOperator

It represents an operation where:

The input type and the output type are the same.

Its functional shape is:

$T \rightarrow T$

For example:

```
UnaryOperator<Integer> square =
    x -> x * x;
```

Here:

Integer $\rightarrow$ Integer

Input:

5

Output:

The important point is:

INPUT TYPE == OUTPUT TYPE

2. Why does UnaryOperator<T> exist?

Without UnaryOperator, you could write:

```
Function<Integer, Integer> square =  
    x -> x * x;
```

That works.

But because the input and output types are the same, Java provides the more specialized interface:

```
UnaryOperator<Integer> square =  
    x -> x * x;
```

So:

Function<T,R>

is general:

$T \rightarrow R$

while:

UnaryOperator<T>

is specialized:

$T \rightarrow T$

Think of it as:

UnaryOperator = Function where $T == R$.

3. Structure of UnaryOperator<T>

Conceptually, its declaration is:

@FunctionalInterface

```

public interface UnaryOperator<T>
    extends Function<T, T> {

    static <T> UnaryOperator<T> identity() {
        return t -> t;
    }
}

```

Notice something very important:

UnaryOperator extends:

Function<T,T>

Therefore it inherits the functionality of Function.

The hierarchy is:

```

Function<T,T>
  ▲
  |
UnaryOperator<T>

```

Consequently, UnaryOperator<T> gets:

```

apply()
compose()
andThen()

```

from Function, and defines:

```
identity()
```

as a static method.

4. apply() — the main operation

The main abstract method is:

```
R apply(T t);
```

inherited from Function<T,T>.

Because UnaryOperator<T> fixes both types to T, it becomes:

```
T apply(T t);
```

For example:

```
UnaryOperator<Integer> doubleValue =  
    x -> x * 2;
```

```
System.out.println(  
    doubleValue.apply(10)  
);
```

Output:

20

Execution:

```
10  
↓  
apply(10)  
↓  
10 * 2  
↓  
20
```

5. Why is apply() abstract?

Because UnaryOperator doesn't know what operation you want to perform.

You could create:

Square

```
UnaryOperator<Integer> square =  
    x -> x * x;
```

Increment

```
UnaryOperator<Integer> increment =  
    x -> x + 1;
```

Uppercase

```
UnaryOperator<String> upper =  
    s -> s.toUpperCase();
```

Trim

```
UnaryOperator<String> trim =
```

```
s -> s.trim();
```

All of these have the same shape:

$T \rightarrow T$

but perform different operations.

6. Basic example

```
import java.util.function.UnaryOperator;

public class UnaryOperatorDemo {

    public static void main(String[] args) {

        UnaryOperator<Integer> square =
            x -> x * x;

        int result =
            square.apply(5);

        System.out.println(result);
    }
}
```

Output:

25

Flow:

```
5
↓
UnaryOperator
↓
apply()
↓
5 * 5
↓
25
```

7. UnaryOperator<String>

UnaryOperator isn't restricted to numbers.

For example:

```
UnaryOperator<String> upperCase =  
    text -> text.toUpperCase();  
  
System.out.println(  
    upperCase.apply("java")  
);
```

Output:

JAVA

The type is:

String → String

Therefore it qualifies as a UnaryOperator.

8. UnaryOperator<Employee>

It can also transform an object into another object of the **same type**.

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

You could write:

```
UnaryOperator<Employee> increaseSalary =  
    employee -> {  
        employee.salary =  
            employee.salary * 1.10;  
  
        return employee;  
    };
```

Then:

Employee emp =

```
        new Employee("Anurag", 50000);  
increaseSalary.apply(emp);  
System.out.println(emp.salary);
```

Output:

55000.0

The transformation is:

```
Employee  
↓  
modify Employee  
↓  
Employee
```

The input and output types remain:

Employee

9. identity() — the identity operation

identity() is a **static method**.

It returns a UnaryOperator that:

Returns its input unchanged.

Example:

```
UnaryOperator<String> identity =  
    UnaryOperator.identity();  
  
System.out.println(  
    identity.apply("Java")  
);
```

Output:

Java

Conceptually:

x -> x

So:

`UnaryOperator.identity()`

is essentially a reusable version of:

`x -> x`

10. Why is it called identity?

In mathematics, an identity operation leaves a value unchanged.

For example:

```
identity(10) = 10  
identity("Java") = "Java"
```

Therefore:

```
Input  
↓  
identity  
↓  
same Input
```

Mathematically:

$f(x) = x$

11. `identity()` example with Integer

```
UnaryOperator<Integer> identity =  
    UnaryOperator.identity();
```

```
System.out.println(  
    identity.apply(100)  
);
```

Output:

100

The flow is:

```
100  
↓
```

identity

↓

100

No transformation occurs.

12. identity() is static, not default

This is an important interview point.

You call:

```
UnaryOperator.identity();
```

not:

```
operator.identity();
```

because identity() is a static method.

Therefore:

apply()	→ abstract
compose()	→ default, inherited
andThen()	→ default, inherited
identity()	→ static

13. compose() and andThen()

Although you specifically mentioned apply() and identity(), UnaryOperator also inherits the composition methods from Function.

These are extremely important.

Suppose:

```
UnaryOperator<Integer> add10 =  
    x -> x + 10;
```

```
UnaryOperator<Integer> multiply2 =  
    x -> x * 2;
```

You can compose them.

14. andThen()

Consider:

```
UnaryOperator<Integer> add10 =  
    x -> x + 10;
```

```
UnaryOperator<Integer> multiply2 =  
    x -> x * 2;
```

```
UnaryOperator<Integer> operation =  
    add10.andThen(multiply2);
```

Then:

```
System.out.println(  
    operation.apply(5)  
);
```

Execution:

```
5  
↓  
add10  
↓  
15  
↓  
multiply2  
↓  
30
```

Output:

30

So:

```
add10.andThen(multiply2)
```

means:

first add 10
then multiply by 2

Conceptually:

```
x -> multiply2.apply(  
    add10.apply(x)  
)
```

15. compose()

compose() executes the supplied operation first.

Using the same operators:

```
UnaryOperator<Integer> add10 =  
    x -> x + 10;
```

```
UnaryOperator<Integer> multiply2 =  
    x -> x * 2;
```

Now:

```
UnaryOperator<Integer> operation =  
    add10.compose(multiply2);
```

Call:

```
operation.apply(5);
```

Execution:

```
5  
↓  
multiply2  
↓  
10  
↓  
add10  
↓  
20
```

Output:

20

So:

```
add10.compose(multiply2)
```

means:

```
first multiply2  
then add10
```

16. andThen() vs compose()

This is an important interview question.

Suppose:

```
UnaryOperator<Integer> f =  
    x -> x + 10;
```

```
UnaryOperator<Integer> g =  
    x -> x * 2;
```

f.andThen(g)

f.andThen(g)

means:

$f \rightarrow g$

or:

```
x  
↓  
x + 10  
↓  
* 2
```

For x = 5:

$5 \rightarrow 15 \rightarrow 30$

f.compose(g)

f.compose(g)

means:

$g \rightarrow f$

or:

```
x  
↓  
* 2  
↓  
+ 10
```

For x = 5:

5 → 10 → 20

Mnemonic:

andThen = me first, then the other
compose = the other first, then me

17. Internal logic of andThen()

Because UnaryOperator extends Function, it inherits Function.andThen().

Conceptually:

```
default <V> Function<T,V> andThen(  
    Function<? super R, ? extends V> after) {  
  
    return (T t) ->  
        after.apply(apply(t));  
}
```

For:

R = T

in UnaryOperator<T>, the idea becomes:

```
x  
↓  
this.apply(x)  
↓  
after.apply(result)  
↓  
result
```

Example:

```
UnaryOperator<Integer> add =  
    x -> x + 10;
```

```
UnaryOperator<Integer> multiply =  
    x -> x * 2;
```

Then:

```
add.andThen(multiply)
```

conceptually means:

```
x -> multiply.apply(  
    add.apply(x)  
)
```

18. Internal logic of compose()

Conceptually:

```
default <V> Function<V,R> compose(  
    Function<? super V, ? extends T> before) {  
  
    return (V v) ->  
        apply(before.apply(v));  
}
```

For a UnaryOperator:

```
x  
↓  
before.apply(x)  
↓  
this.apply(result)  
↓  
result
```

So:

```
add.compose(multiply)
```

conceptually means:

```
x -> add.apply(  
    multiply.apply(x)  
)
```

19. identity() internal concept

The identity method is conceptually:

```
static <T> UnaryOperator<T> identity() {  
    return t -> t;  
}
```

Therefore:

```
UnaryOperator<String> identity =  
    UnaryOperator.identity();
```

is conceptually equivalent to:

```
UnaryOperator<String> identity =  
    s -> s;
```

For Integer:

```
UnaryOperator<Integer> identity =  
    x -> x;
```

20. Why identity() is useful

At first glance:

```
x -> x
```

looks useless.

But it is valuable when an API requires a transformation function and sometimes you don't want to transform the value.

For example, you may have:

```
UnaryOperator<String> operation =  
    UnaryOperator.identity();
```

Then:

```
String result =  
    operation.apply("Java");
```

The value passes through unchanged.

This is especially useful in generic algorithms where you need a valid operation but don't want to modify the data.

21. UnaryOperator with Stream API

Suppose:

```
List<Integer> numbers =  
    Arrays.asList(1, 2, 3, 4, 5);
```

Create:

```
UnaryOperator<Integer> doubleValue =  
    x -> x * 2;
```

Use it:

```
numbers.stream()  
    .map(doubleValue)  
    .forEach(System.out::println);
```

Output:

```
2  
4  
6  
8  
10
```

The relationship is:

```
Stream element  
↓  
UnaryOperator.apply()  
↓  
transformed element  
↓  
map()
```

22. UnaryOperator with List.replaceAll()

A particularly good real-world example is:

```
List<String> names =  
    new ArrayList<>(  
        Arrays.asList(  
            "amit",  
            "rahul",  
            "anurag"  
        )  
    );
```

Create:

```
UnaryOperator<String> upperCase =  
    String::toUpperCase;
```

Then:

```
names.replaceAll(upperCase);
```

The list becomes:

```
[AMIT, RAHUL, ANURAG]
```

Why does this work?

Because:

```
List.replaceAll(UnaryOperator<E> operator)
```

expects an operation:

$E \rightarrow E$

and `UnaryOperator<E>` represents exactly that.

23. `replaceAll()` algorithm

Consider:

```
List<String> names =  
    new ArrayList<>(  
        Arrays.asList(  
            "java",  
            "python",  
            "sql"  
        )  
    );
```

```
UnaryOperator<String> upper =  
    String::toUpperCase;
```

```
names.replaceAll(upper);
```

Conceptually:

```
"java"  
↓  
upper.apply()  
↓  
"JAVA"
```

```
"python"  
↓  
upper.apply()
```

↓
"PYTHON"

"sql"
↓
upper.apply()
↓
"SQL"

Final list:

[JAVA, PYTHON, SQL]

24. UnaryOperator and method references

UnaryOperator is frequently used with method references.

For example:

```
UnaryOperator<String> upper =  
    String::toUpperCase;
```

This is equivalent to:

```
UnaryOperator<String> upper =  
    s -> s.toUpperCase();
```

Both have:

String → String

Another:

```
UnaryOperator<String> trim =  
    String::trim;
```

Equivalent to:

```
UnaryOperator<String> trim =  
    s -> s.trim();
```

25. UnaryOperator and constructor references

A constructor reference can be used when the constructor accepts the same type and returns that same type, although this pattern is less common.

For example, imagine a constructor:

```
class Employee {  
    Employee(Employee other) {  
        // copy  
    }  
}
```

Then:

```
UnaryOperator<Employee> copy =  
    Employee::new;
```

The shape is:

$\text{Employee} \rightarrow \text{Employee}$

Therefore it matches:

```
UnaryOperator<Employee>
```

26. UnaryOperator vs Function

This is one of the most important comparisons.

Function

```
Function<String, Integer> length =  
    s -> s.length();
```

Shape:

$\text{String} \rightarrow \text{Integer}$

Input and output types are different.

UnaryOperator

```
UnaryOperator<String> upper =  
    s -> s.toUpperCase();
```

Shape:

$\text{String} \rightarrow \text{String}$

Input and output types are the same.

Therefore:

Function<T,R>

is more general.

While:

UnaryOperator<T>

means:

Function<T,T>

27. UnaryOperator vs BinaryOperator

Another important distinction:

UnaryOperator

Takes **one input**:

$T \rightarrow T$

Example:

UnaryOperator<Integer> square =
x -> x * x;

BinaryOperator

Takes **two inputs**:

$(T, T) \rightarrow T$

Example:

BinaryOperator<Integer> add =
(a, b) -> a + b;

Call:

add.apply(10, 20);

Result:

30

Therefore:

UnaryOperator
↓
one operand
↓
 $T \rightarrow T$
BinaryOperator
↓
two operands
↓
 $(T,T) \rightarrow T$

28. UnaryOperator vs Consumer

UnaryOperator

```
UnaryOperator<Integer> doubleValue =  
    x -> x * 2;
```

Returns another Integer:

$5 \rightarrow 10$

Consumer

```
Consumer<Integer> print =  
    x -> System.out.println(x);
```

Returns nothing:

$5 \rightarrow \text{action} \rightarrow \text{void}$

So:

UnaryOperator:
 $T \rightarrow T$

Consumer:
 $T \rightarrow \text{void}$

29. UnaryOperator vs Predicate

UnaryOperator

```
UnaryOperator<Integer> increment =
```

```
    x -> x + 1;  
10 → 11
```

Predicate

```
Predicate<Integer> positive =  
    x -> x > 0;  
10 → true
```

Therefore:

UnaryOperator:
 $T \rightarrow T$

Predicate:
 $T \rightarrow \text{boolean}$

30. Complete example using apply(), identity(), andThen(), and compose()

```
import java.util.function.UnaryOperator;  
  
public class UnaryOperatorDemo {  
  
    public static void main(String[] args) {  
  
        // apply()  
        UnaryOperator<Integer> square =  
            x -> x * x;  
  
        System.out.println(  
            square.apply(5)  
        );  
  
        // identity()  
        UnaryOperator<Integer> identity =  
            UnaryOperator.identity();  
  
        System.out.println(  
            identity.apply(5)  
        );  
  
        // andThen()  
        UnaryOperator<Integer> add10 =  
            x -> x + 10;  
  
        UnaryOperator<Integer> multiply2 =  
            x -> x * 2;
```

```

UnaryOperator<Integer> andThenOperation =
    add10.andThen(multiply2);

System.out.println(
    andThenOperation.apply(5)
);

// compose()
UnaryOperator<Integer> composeOperation =
    add10.compose(multiply2);

System.out.println(
    composeOperation.apply(5)
);
}
}

```

Output:

```

25
5
30
20

```

Explanation:

```

square.apply(5)
↓
25
identity.apply(5)
↓
5
add10.andThen(multiply2)
5 → 15 → 30
add10.compose(multiply2)
5 → 10 → 20

```

31. Important method classification

This is the most accurate way to describe the methods:

Method	Where defined	Type	Meaning
apply(T)	Function<T,T>	Abstract	Transform one value into another same-type value
andThen()	Function	Default	Apply this operation, then another

compose()	Function	Default	Apply another operation, then this
identity()	UnaryOperator	Static	Return input unchanged

So saying:

"UnaryOperator includes default methods such as apply() and identity()"

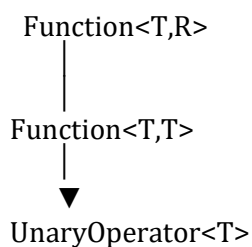
is technically incorrect.

The correct statement is:

UnaryOperator<T> inherits the abstract apply(T) method and the default compose() and andThen() methods from Function<T,T>, and provides the static identity() method.

32. Relationship between Function and UnaryOperator

This relationship is very important:



More precisely:

UnaryOperator<T>
extends
Function<T,T>

Therefore:

UnaryOperator<Integer>

is essentially a specialized:

Function<Integer, Integer>

33. Why does Java provide UnaryOperator if Function already exists?

Because it communicates intent more clearly.

Compare:

```
Function<Integer, Integer> operation =  
    x -> x * 2;
```

versus:

```
UnaryOperator<Integer> operation =  
    x -> x * 2;
```

The second immediately tells the reader:

"This operation receives an Integer and produces another Integer."

It is specifically an **operator on the same type**.

This is especially useful with APIs such as:

List.replaceAll()

and stream transformations.

34. Internal algorithm

Consider:

```
UnaryOperator<Integer> operation =  
    x -> x * 2;
```

Then:

```
operation.apply(10);
```

The conceptual algorithm is:

1. Call apply(10)
↓
2. Lambda receives x = 10
↓
3. Evaluate x * 2
↓
4. Evaluate 10 * 2
↓
5. Produce 20

↓
6. Return Integer 20

So:

10
↓
apply()
↓
x * 2
↓
20

35. Identity algorithm

For:

```
UnaryOperator<Integer> identity =  
    UnaryOperator.identity();
```

then:

```
identity.apply(10);
```

conceptually:

10
↓
identity
↓
return same 10

Equivalent lambda:

```
x -> x
```

Algorithm:

Input
↓
No transformation
↓
Same input

36. Interview-ready answer

If asked:

What is UnaryOperator<T>?

UnaryOperator<T> is a Java functional interface that represents an operation on a single operand where the input and output are of the same type.

Its functional shape is:

$T \rightarrow T$

What is its abstract method?

It inherits:

`T apply(T t);`

from:

`Function<T,T>`

What is identity()?

`identity()` is a **static method** that returns a `UnaryOperator` that returns its input unchanged.

```
UnaryOperator<Integer> identity =  
    UnaryOperator.identity();
```

Conceptually:

$x \rightarrow x$

Does UnaryOperator have default methods?

Yes, through `Function<T,T>` it inherits:

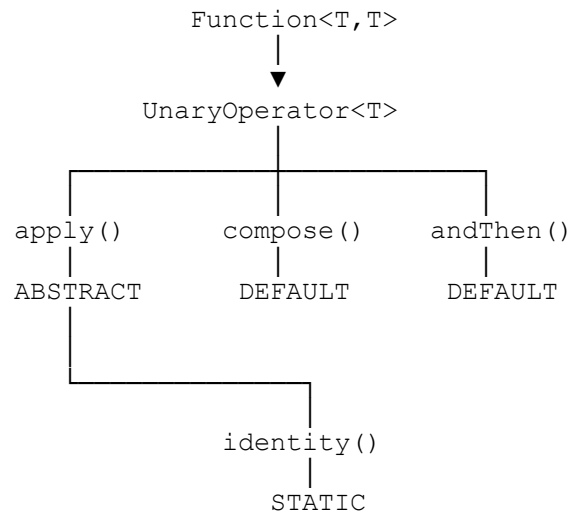
```
compose()  
andThen()
```

Both are default methods.

What is the difference between Function and UnaryOperator?

```
Function<T,R>  → T → R  
UnaryOperator<T> → T → T
```

37. Final mental model



And the easiest mnemonic is:

`UnaryOperator` = ONE input → SAME TYPE output

<code>apply()</code>	= perform transformation
<code>compose()</code>	= other operation first
<code>andThen()</code>	= this operation first
<code>identity()</code>	= return input unchanged

For example:

```
UnaryOperator<Integer> operation =  
    x -> x * 2;
```

means:

```
Integer  
↓  
apply()  
↓  
Integer
```

while:

```
UnaryOperator<Integer> identity =  
    UnaryOperator.identity();
```

means:

```
Integer
```

↓
identity
↓
same Integer

So the key correction to your original statement is:

apply() → ABSTRACT (inherited from Function)
compose() → DEFAULT (inherited from Function)
andThen() → DEFAULT (inherited from Function)
identity() → STATIC

This inheritance relationship is the key to fully understanding UnaryOperator<T>.

Yes. There is an important correction to your statement:

BinaryOperator<T> is a functional interface whose abstract method is apply(T, T). minBy() and maxBy() are static methods, not default methods.

Also, unlike UnaryOperator, BinaryOperator is a specialization of BiFunction<T,T,T>.

The technically accurate structure is:

```
BinaryOperator<T>
├── apply(T, T) → abstract
├── minBy()    → static
└── maxBy()    → static
```

And because it extends BiFunction<T,T,T>, it inherits the BiFunction methods such as andThen().

1. What is BinaryOperator<T>?

BinaryOperator<T> is a built-in functional interface in:

java.util.function.BinaryOperator

It represents:

An operation that accepts two operands of the same type and returns a result of that same type.

Its functional shape is:

$(T, T) \rightarrow T$

For example:

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

Here:

$\text{Integer} + \text{Integer} \rightarrow \text{Integer}$

Example:

```
System.out.println(add.apply(10, 20));
```

Output:

30

2. Why is it called BinaryOperator?

The word **binary** means that the operation works on **two operands**.

For example:

10 + 20

has two operands:

10

20

and one result:

30

So:

10 + 20 → 30

matches:

(T, T) → T

3. BinaryOperator<T> hierarchy

BinaryOperator<T> extends:

BiFunction<T, T, T>

So the hierarchy is:

BiFunction<T,T,T>



BinaryOperator<T>

Compare this with UnaryOperator:

Function<T,T>



|
UnaryOperator<T>

Therefore:

UnaryOperator:
 $T \rightarrow T$

BinaryOperator:
 $(T, T) \rightarrow T$

4. Structure of BinaryOperator<T>

Conceptually:

```
@FunctionalInterface
public interface BinaryOperator<T>
    extends BiFunction<T, T, T> {

    static <T> BinaryOperator<T> minBy(
        Comparator<? super T> comparator) {
        ...
    }

    static <T> BinaryOperator<T> maxBy(
        Comparator<? super T> comparator) {
        ...
    }
}
```

The inherited BiFunction method is:

R apply(T t, U u);

Because:

T = T
U = T
R = T

it becomes:

T apply(T t, T u);

So:

BinaryOperator<T>
|

```
|— apply(T,T) → abstract
|— minBy()   → static
|— maxBy()   → static
```

5. apply() — the main operation

The abstract method is:

```
T apply(T t, T u);
```

It:

Accepts two values of type T and produces one value of type T.

Example:

```
BinaryOperator<Integer> add =
    (a, b) -> a + b;
```

```
int result = add.apply(10, 20);
```

```
System.out.println(result);
```

Output:

```
30
```

Execution:

```
10 —┐
    └─┐ add —> 30
20 —┐
```

6. Why is apply() abstract?

Because BinaryOperator doesn't know what operation you want.

You could define:

Addition

```
BinaryOperator<Integer> add =
    (a, b) -> a + b;
```

Multiplication

```
BinaryOperator<Integer> multiply =  
    (a, b) -> a * b;
```

Maximum

```
BinaryOperator<Integer> maximum =  
    (a, b) -> Math.max(a, b);
```

String concatenation

```
BinaryOperator<String> concatenate =  
    (a, b) -> a + b;
```

All have the same shape:

$$(T, T) \rightarrow T$$

but perform different operations.

7. Basic example

```
import java.util.function.BinaryOperator;  
  
public class BinaryOperatorDemo {  
  
    public static void main(String[] args) {  
  
        BinaryOperator<Integer> add =  
            (a, b) -> a + b;  
  
        System.out.println(  
            add.apply(10, 20)  
        );  
    }  
}
```

Output:

30

Algorithm:

1. Receive first value = 10
2. Receive second value = 20
3. Execute $a + b$
4. Calculate $10 + 20$
5. Return 30

8. BinaryOperator with String

A BinaryOperator isn't limited to arithmetic.

```
BinaryOperator<String> concatenate =  
    (a, b) -> a + " " + b;
```

```
System.out.println(  
    concatenate.apply("Hello", "Java")  
);
```

Output:

Hello Java

Type flow:

String + String → String

9. BinaryOperator with custom objects

Suppose:

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

You can define:

```
BinaryOperator<Employee> higherSalary =  
    (e1, e2) ->  
        e1.salary >= e2.salary  
        ? e1  
        : e2;
```

Now:

```
Employee e1 =  
    new Employee("Amit", 50000);
```

```
Employee e2 =  
    new Employee("Rahul", 70000);
```

```
Employee result =  
    higherSalary.apply(e1, e2);
```

```
System.out.println(result.name);
```

Output:

Rahul

The operation is:

```
Employee  
+  
Employee  
↓  
Employee
```

10. minBy() — static method

Now we come to minBy().

First, the correction:

minBy() is a static method, not a default method.

Its purpose is:

Create a BinaryOperator<T> that returns the smaller of two values according to a supplied Comparator.

Conceptually:

```
BinaryOperator<T> minBy(  
    Comparator<? super T> comparator)
```

11. minBy() with Integers

Example:

```
BinaryOperator<Integer> minimum =  
    BinaryOperator.minBy(  
        Integer::compare
```

```
);
```

Then:

```
System.out.println(  
    minimum.apply(10, 20)  
);
```

Output:

10

Execution:

```
10 ┌───  
   │   └─── Comparator ──> smaller = 10  
20 ┌───
```

12. minBy() with a Comparator

You can explicitly create the Comparator:

```
Comparator<Integer> comparator =  
    (a, b) -> Integer.compare(a, b);  
  
BinaryOperator<Integer> minimum =  
    BinaryOperator.minBy(comparator);
```

Then:

```
minimum.apply(50, 20);
```

returns:

20

13. minBy() conceptual implementation

Conceptually, it behaves approximately like:

```
static <T> BinaryOperator<T> minBy(  
    Comparator<? super T> comparator) {  
  
    return (a, b) ->  
        comparator.compare(a, b) <= 0  
        ? a
```

```
        : b;  
    }
```

Therefore:

```
BinaryOperator<Integer> min =  
    BinaryOperator.minBy(  
        Integer::compare  
    );
```

can be thought of as:

```
Input A  
+  
Input B  
↓  
Comparator  
↓  
Which one is smaller?  
↓  
Return smaller value
```

14. `maxBy()` — static method

`maxBy()` is also a:

static method

not a default method.

Its purpose is:

Create a `BinaryOperator<T>` that returns the larger of two values according to a `Comparator`.

Example:

```
BinaryOperator<Integer> maximum =  
    BinaryOperator.maxBy(  
        Integer::compare  
    );  
  
System.out.println(  
    maximum.apply(10, 20)  
);
```

Output:

15. maxBy() conceptual implementation

Conceptually:

```
static <T> BinaryOperator<T> maxBy(  
    Comparator<? super T> comparator) {  
  
    return (a, b) ->  
        comparator.compare(a, b) >= 0  
            ? a  
            : b;  
}
```

So:

```
A  
+  
B  
↓  
Comparator  
↓  
Which is larger?  
↓  
Return larger
```

16. minBy() vs maxBy()

Method Type Purpose

minBy() Static Returns smaller value

maxBy() Static Returns larger value

Example:

```
BinaryOperator<Integer> min =  
    BinaryOperator.minBy(Integer::compare);
```

```
BinaryOperator<Integer> max =  
    BinaryOperator.maxBy(Integer::compare);
```

Then:

```
System.out.println(min.apply(10, 20));  
System.out.println(max.apply(10, 20));
```

Output:

10
20

17. Why does `minBy()` need a Comparator?

Because Java needs to know **how to compare the objects**.

For integers:

`Integer::compare`

is obvious.

But consider:

Employee

What does "minimum employee" mean?

Could it mean:

- lowest salary?
- lowest age?
- alphabetically smallest name?
- earliest joining date?

You must tell Java how to compare them.

18. `minBy()` with Employee salary

Suppose:

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

Create a Comparator:

```
Comparator<Employee> salaryComparator =  
    (e1, e2) ->  
        Double.compare(  
            e1.salary,  
            e2.salary  
        );
```

Create the BinaryOperator:

```
BinaryOperator<Employee> lowestSalary =  
    BinaryOperator.minBy(  
        salaryComparator  
    );
```

Then:

```
Employee e1 =  
    new Employee("Amit", 50000);
```

```
Employee e2 =  
    new Employee("Rahul", 70000);
```

```
Employee result =  
    lowestSalary.apply(e1, e2);
```

```
System.out.println(result.name);
```

Output:

Amit

19. maxBy() with Employee salary

Similarly:

```
BinaryOperator<Employee> highestSalary =  
    BinaryOperator.maxBy(  
        salaryComparator  
    );
```

Then:

```
Employee result =  
    highestSalary.apply(e1, e2);
```

```
System.out.println(result.name);
```

Output:

Rahul

Execution:

```
Amit, 50000
+
Rahul, 70000
↓
salary Comparator
↓
70000 is greater
↓
Rahul
```

20. Very useful combination with Streams

BinaryOperator is particularly important with:

`Stream.reduce()`

For example:

```
List<Integer> numbers =
    Arrays.asList(10, 20, 30, 40);
```

You can calculate the sum:

```
BinaryOperator<Integer> add =
    (a, b) -> a + b;
```

```
int sum =
    numbers.stream()
        .reduce(0, add);
```

Result:

100

The reduction conceptually works like:

```
0 + 10 = 10
10 + 20 = 30
30 + 30 = 60
60 + 40 = 100
```

21. reduce() and BinaryOperator

This is a very important relationship.

reduce() repeatedly combines two values.

That is exactly what a BinaryOperator does:

value + value → value

For example:

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

Then:

```
numbers.stream()  
    .reduce(0, add);
```

The algorithm is:

```
    BinaryOperator  
      ↓  
0 + 10 → 10  
10 + 20 → 30  
30 + 30 → 60  
60 + 40 → 100
```

22. Finding maximum using BinaryOperator

```
List<Integer> numbers =  
    Arrays.asList(10, 50, 20, 90, 30);
```

```
BinaryOperator<Integer> max =  
    BinaryOperator.maxBy(  
        Integer::compare  
    );
```

```
int result =  
    numbers.stream()  
        .reduce(max)  
        .orElseThrow();
```

```
System.out.println(result);
```

Output:

90

The reduction is conceptually:

10 + 50 → 50
50 + 20 → 50
50 + 90 → 90
90 + 30 → 90

Here + doesn't mean arithmetic addition; it represents the BinaryOperator operation:

choose maximum

23. Finding minimum using BinaryOperator

```
BinaryOperator<Integer> min =  
    BinaryOperator.minBy(  
        Integer::compare  
    );
```

```
int result =  
    numbers.stream()  
        .reduce(min)  
        .orElseThrow();
```

```
System.out.println(result);
```

Output:

10

24. BinaryOperator and BiFunction

This distinction is important.

BiFunction<T,U,R> represents:

$(T, U) \rightarrow R$

Example:

```
BiFunction<Integer, Double, String> f =  
    (i, d) -> i + ":" + d;
```

Types:

Integer + Double $\rightarrow$ String

But BinaryOperator<T> represents:

$(T, T) \rightarrow T$

Example:

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

Types:

Integer + Integer $\rightarrow$ Integer

Therefore:

BiFunction<T,U,R>
↓
different types allowed

BinaryOperator<T>
↓
both inputs and output same type

25. UnaryOperator vs BinaryOperator

This is another important Java 8 interview question.

UnaryOperator

One input:

$T \rightarrow T$

Example:

```
UnaryOperator<Integer> square =  
    x -> x * x;  
5  $\rightarrow$  25
```

BinaryOperator

Two inputs:

$(T,T) \rightarrow T$

Example:

```
BinaryOperator<Integer> add =  
    (a,b) -> a + b;  
5 + 10 → 15
```

Comparison:

Interface	Number of inputs	Output	Shape
UnaryOperator<T>	1	Same type	$T \rightarrow T$
BinaryOperator<T>	2	Same type	$(T,T) \rightarrow T$

26. BinaryOperator vs Function

Function

```
Function<Integer, String> convert =  
    x -> "Value = " + x;
```

Shape:

$\text{Integer} \rightarrow \text{String}$

BinaryOperator

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

Shape:

$(\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$

So:

Function:
one input $\rightarrow$ potentially different output

BinaryOperator:
two same-type inputs $\rightarrow$ same-type output

27. BinaryOperator vs Predicate

Predicate

```
Predicate<Integer> greater =  
    x -> x > 10;
```

Shape:

$\text{Integer} \rightarrow \text{boolean}$

BinaryOperator

```
BinaryOperator<Integer> add =  
    (a,b) -> a + b;
```

Shape:

$(\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$

Therefore:

Predicate:
 $T \rightarrow \text{boolean}$

BinaryOperator:
 $(T, T) \rightarrow T$

28. BinaryOperator vs Consumer

Consumer

```
Consumer<Integer> print =  
    x -> System.out.println(x);
```

Shape:

$\text{Integer} \rightarrow \text{void}$

BinaryOperator

```
BinaryOperator<Integer> add =  
    (a,b) -> a + b;
```

Shape:

$(\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$

29. Inherited andThen()

Because:

```
BinaryOperator<T>  
    extends  
BiFunction<T,T,T>
```

it inherits BiFunction's default:

```
andThen()
```

This is an important point.

For example:

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

```
Function<Integer, String> format =  
    x -> "Result = " + x;
```

You can write:

```
Function<Integer, String> result =  
    add.andThen(format);
```

Then:

```
System.out.println(  
    result.apply(10, 20)  
);
```

Output:

Result = 30

Execution:

```
10 + 20  
↓  
30  
↓  
format  
↓  
"Result = 30"
```

Notice that the result of the complete operation is now a Function, because `andThen()` can change the output type.

30. Important method classification

This is the technically accurate table:

Method	Defined/inherited from	Type	Purpose
<code>apply(T,T)</code>	<code>BiFunction<T,T,T></code>	Abstract	Combine two values
<code>andThen()</code>	<code>BiFunction</code>	Default	Apply another function afterward
<code>minBy()</code>	<code>BinaryOperator</code>	Static	Create operator selecting minimum
<code>maxBy()</code>	<code>BinaryOperator</code>	Static	Create operator selecting maximum

So your original statement:

"`BinaryOperator` functional interfaces include default methods such as `apply()`, `minBy()`, `maxBy()`"

should be corrected to:

`BinaryOperator<T>` is a functional interface that inherits the abstract `apply(T,T)` method and default `andThen()` method from `BiFunction<T,T,T>`, and provides the static `minBy()` and `maxBy()` methods.

31. Complete example

```
import java.util.function.BinaryOperator;
```

```
public class BinaryOperatorDemo {
```

```
    public static void main(String[] args) {
```

```
        // apply()
        BinaryOperator<Integer> add =
            (a, b) -> a + b;
```

```
        System.out.println(
```

```

        add.apply(10, 20)
    );

    // minBy()
    BinaryOperator<Integer> min =
        BinaryOperator.minBy(
            Integer::compare
        );

    System.out.println(
        min.apply(10, 20)
    );

    // maxBy()
    BinaryOperator<Integer> max =
        BinaryOperator.maxBy(
            Integer::compare
        );

    System.out.println(
        max.apply(10, 20)
    );
}
}

```

Output:

```

30
10
20

```

32. Complete Employee example

```

import java.util.function.BinaryOperator;

class Employee {

    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return name + " : " + salary;
    }
}

```

```

    }
}

public class EmployeeBinaryOperatorDemo {

    public static void main(String[] args) {

        Employee e1 =
            new Employee("Amit", 50000);

        Employee e2 =
            new Employee("Rahul", 70000);

        BinaryOperator<Employee> higherSalary =
            BinaryOperator.maxBy(
                (a, b) ->
                    Double.compare(
                        a.salary,
                        b.salary
                    )
            );

        BinaryOperator<Employee> lowerSalary =
            BinaryOperator.minBy(
                (a, b) ->
                    Double.compare(
                        a.salary,
                        b.salary
                    )
            );

        System.out.println(
            higherSalary.apply(e1, e2)
        );

        System.out.println(
            lowerSalary.apply(e1, e2)
        );
    }
}

```

Output:

Rahul : 70000.0
Amit : 50000.0

33. How maxBy() works in the Employee example

For:

higherSalary.apply(e1, e2)

the algorithm is:

Employee e1
salary = 50000

Employee e2
salary = 70000

↓

Comparator.compare(e1, e2)

↓

50000 compared with 70000

↓

70000 is greater

↓

return e2

↓

Rahul

The important thing is that maxBy() doesn't know anything about salaries.

The **Comparator** defines what "maximum" means.

34. Different Comparator = different maximum

Suppose the same Employee objects have:

Amit → salary 50000
Rahul → salary 70000

Maximum by salary:

```
BinaryOperator<Employee> maxSalary =  
    BinaryOperator.maxBy(  
        Comparator.comparingDouble(  
            e -> e.salary  
        )  
    );
```

Result:

Rahul

But maximum by name:

```
BinaryOperator<Employee> maxName =  
    BinaryOperator.maxBy(  
        Comparator.comparing(  
            e -> e.name  
        )  
    );
```

The result could be:

Rahul

The BinaryOperator remains the same concept:

(Employee, Employee) → Employee

Only the comparison rule changes.

35. BinaryOperator with reduce()

This is probably the most important practical application.

Consider:

```
List<Integer> numbers =  
    Arrays.asList(1, 2, 3, 4, 5);
```

Create:

```
BinaryOperator<Integer> sum =  
    (a, b) -> a + b;
```

Then:

```
int result =  
    numbers.stream()  
        .reduce(0, sum);
```

Result:

15

Conceptually:

$0 + 1 = 1$
 $1 + 2 = 3$
 $3 + 3 = 6$
 $6 + 4 = 10$
 $10 + 5 = 15$

This is exactly the type of repeated operation that `BinaryOperator` represents.

36. `BinaryOperator` and `Stream.reduce()`

Think about `reduce()` like this:

Many values
↓
combine two at a time
↓
one final value

For example:

10
20
30
40

using addition:

$10 + 20 \rightarrow 30$
 $30 + 30 \rightarrow 60$
 $60 + 40 \rightarrow 100$

The combining operation:

$(a,b) \rightarrow \text{result}$

is naturally represented by:

BinaryOperator<Integer>

37. Why BinaryOperator is useful in parallel streams

A BinaryOperator is especially relevant to reduction operations because a stream can potentially combine partial results.

For example:

10 20 30 40

could be divided conceptually:

$10 + 20 \rightarrow 30$

$30 + 40 \rightarrow 70$

$30 + 70 \rightarrow 100$

The combining operation must be appropriate for reduction, particularly when using parallel streams.

For operations such as addition, multiplication, minimum, and maximum, this works naturally.

38. minBy() and maxBy() with Stream reduction

Example:

```
List<Integer> numbers =  
    Arrays.asList(  
        50, 20, 90, 10, 70  
    );
```

Maximum:

```
BinaryOperator<Integer> max =  
    BinaryOperator.maxBy(  
        Integer::compare  
    );
```

```
Integer highest =  
    numbers.stream()  
        .reduce(max)  
        .orElse(null);
```

Result:

90

Minimum:

```
BinaryOperator<Integer> min =  
    BinaryOperator.minBy(  
        Integer::compare  
    );
```

```
Integer lowest =  
    numbers.stream()  
        .reduce(min)  
        .orElse(null);
```

Result:

10

39. Comparison with Math.max()

You might ask:

Why use BinaryOperator.maxBy() when Java already has Math.max()?

Math.max() directly calculates the maximum of primitive numeric values.

```
Math.max(10, 20);
```

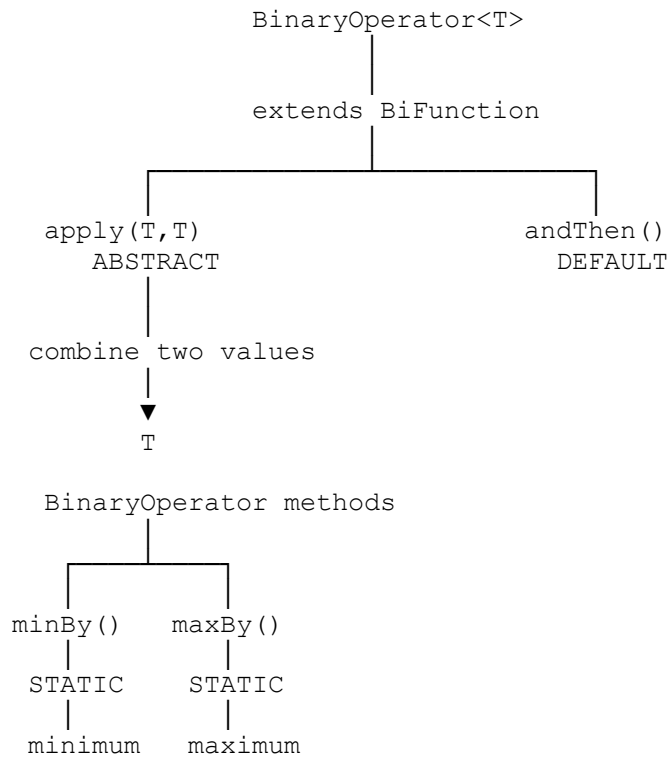
But BinaryOperator.maxBy() works with objects and a Comparator.

For example:

```
BinaryOperator<Employee> maxSalary =  
    BinaryOperator.maxBy(  
        Comparator.comparingDouble(  
            e -> e.salary  
        )  
    );
```

This lets you define "maximum" for arbitrary objects.

40. Final mental model



The easiest mnemonic is:

`BinaryOperator` = TWO inputs → SAME TYPE output

For example:

`BinaryOperator<Integer> add =`
`(a, b) -> a + b;`

means:

`Integer + Integer → Integer`

And:

`BinaryOperator.minBy(comparator)`

means:

two T values

↓

Comparator

↓

return smaller T

while:

BinaryOperator.maxBy(comparator)

means:

two T values

↓

Comparator

↓

return larger T

Final classification

BinaryOperator<T>

- ├── apply(T,T) → ABSTRACT
- ├── andThen() → DEFAULT, inherited from BiFunction
- ├── minBy() → STATIC
- └── maxBy() → STATIC

So the key correction to your original statement is:

apply() is not a default method, and minBy()/maxBy() are not default methods. apply() is the abstract functional method inherited from BiFunction; minBy() and maxBy() are static factory methods defined by BinaryOperator.

Yes. There is an important correction first:

BiFunction<T,U,R> is a functional interface whose abstract method is apply(T,U). apply() is not a default method. andThen() is a default method.

The technically correct structure is:

```
BiFunction<T,U,R>
├── apply(T,U) → abstract
└── andThen() → default
```

Its fundamental functional shape is:

$(T, U) \rightarrow R$

That means:

Take two input values, possibly of different types, and produce one result value, which can also be a different type.

1. What is BiFunction<T,U,R>?

BiFunction<T,U,R> is a built-in functional interface in:

java.util.function.BiFunction

It represents a function that:

- accepts **two arguments**
- processes them
- returns **one result**

Its general form is:

$T \times U \rightarrow R$

or simply:

$(T, U) \rightarrow R$

For example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

Here:

$\text{Integer} + \text{Integer} \rightarrow \text{Integer}$

Another example:

```
BiFunction<String, Integer, String> repeat =  
    (text, count) -> text.repeat(count);
```

Here:

$\text{String} + \text{Integer} \rightarrow \text{String}$

So unlike `BinaryOperator<T>`, the two input types and output type **do not have to be the same**.

2. Structure of BiFunction

Conceptually:

```
@FunctionalInterface  
public interface BiFunction<T, U, R> {  
  
    R apply(T t, U u);  
  
    default <V> BiFunction<T, U, V> andThen(  
        Function<? super R, ? extends V> after) {  
        ...  
    }  
}
```

Therefore:

```
BiFunction<T,U,R>  
├── apply(T,U) → abstract  
└── andThen() → default
```

This is why `BiFunction` is a functional interface: it has exactly **one abstract method**.

3. apply() — the abstract method

The main method is:

```
R apply(T t, U u);
```

It accepts two values and returns one value.

Example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

```
int result = add.apply(10, 20);
```

```
System.out.println(result);
```

Output:

30

Execution:

```
10 ┌───> apply() ──> 30  
20 └───
```

The algorithm is:

1. Receive first input
 2. Receive second input
 3. Execute lambda
 4. Calculate result
 5. Return result
-

4. Why is apply() abstract?

Because BiFunction cannot know what operation you want to perform.

For example:

Addition

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

Multiplication

```
BiFunction<Integer, Integer, Integer> multiply =  
    (a, b) -> a * b;
```

String formatting

```
BiFunction<String, Integer, String> format =  
    (name, age) ->  
        name + " is " + age + " years old";
```

Combining different types

```
BiFunction<String, Integer, String> repeat =  
    (text, count) ->  
        text.repeat(count);
```

All of these require:

`apply(T, U)`

but implement different business logic.

5. Basic BiFunction example

```
import java.util.function.BiFunction;  
  
public class BiFunctionDemo {  
  
    public static void main(String[] args) {  
  
        BiFunction<Integer, Integer, Integer> add =  
            (a, b) -> a + b;  
  
        int result =  
            add.apply(10, 20);  
  
        System.out.println(result);  
    }  
}
```

Output:

30

Flow:

```
10  
+  
20  
↓  
apply()
```

↓
30

6. Different input types

One of the biggest advantages of BiFunction is that T and U can be different types.

For example:

```
BiFunction<String, Integer, String> createMessage =  
    (name, age) ->  
        name + " is " + age + " years old";
```

Call:

```
String result =  
    createMessage.apply("Anurag", 30);
```

```
System.out.println(result);
```

Output:

Anurag is 30 years old

The type relationship is:

String + Integer → String

This would not be directly expressible by BinaryOperator<String> because BinaryOperator requires:

(String, String) → String

7. BiFunction type parameters

The three generic parameters are:

T = first input type
U = second input type
R = return type

For:

```
BiFunction<String, Integer, Double>
```

we have:

```
T = String
U = Integer
R = Double
```

Therefore:

```
R apply(T t, U u)
```

becomes:

```
Double apply(String t, Integer u)
```

Conceptually:

```
String + Integer
    ↓
  BiFunction
    ↓
    Double
```

8. andThen() — the default method

andThen() is the **default method** of BiFunction.

Its purpose is:

Take the result produced by the current BiFunction and pass that result to another Function.

This is different from BinaryOperator, where the two inputs have the same type.

Suppose:

```
BiFunction<Integer, Integer, Integer> add =
    (a, b) -> a + b;
```

and:

```
Function<Integer, Integer> square =
    x -> x * x;
```

Now:

```
BiFunction<Integer, Integer, Integer> operation =
    add.andThen(square);
```

Then:

```
operation.apply(10, 20);
```

executes:

10 + 20

↓

30

↓

30 × 30

↓

900

Output:

900

9. andThen() execution algorithm

Consider:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

```
Function<Integer, Integer> square =  
    x -> x * x;
```

```
BiFunction<Integer, Integer, Integer> combined =  
    add.andThen(square);
```

Calling:

```
combined.apply(10, 20);
```

works conceptually like:

```
square.apply(  
    add.apply(10, 20)  
);
```

Step by step:

Step 1:

10, 20

↓

Step 2:

add.apply(10,20)

↓

Step 3:

30

↓

Step 4:

square.apply(30)

↓

Step 5:

900

So:

BiFunction

↓

produces R

↓

Function<R,V>

↓

produces V

10. Conceptual implementation of andThen()

The default method can be understood approximately as:

```
default <V> BiFunction<T, U, V> andThen(  
    Function<? super R, ? extends V> after) {  
  
    Objects.requireNonNull(after);  
  
    return (T t, U u) ->  
        after.apply(  
            apply(t, u)  
        );  
}
```

This is the key implementation concept.

For:

BiFunction<Integer, Integer, Integer> add

and:

Function<Integer, Integer> square

the combined operation is essentially:

```
(a, b) ->
  square.apply(
    add.apply(a, b)
  );
```

11. Important: andThen() does not combine two BiFunctions

This is a common misunderstanding.

You might think:

```
biFunction1.andThen(biFunction2)
```

means:

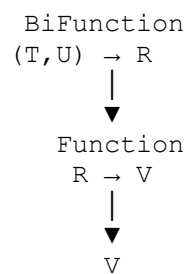
$\text{BiFunction} \rightarrow \text{BiFunction}$

But the `andThen()` method expects a:

Function

that operates on the **result of the first BiFunction**.

Conceptually:



Therefore:

$(T, U) \rightarrow R \rightarrow V$

12. Example with different output type

This is where `andThen()` becomes particularly powerful.

Suppose:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

The result is:

Integer

Now define:

```
Function<Integer, String> format =  
    x -> "Result = " + x;
```

The result is:

String

Combine:

```
BiFunction<Integer, Integer, String> operation =  
    add.andThen(format);
```

Then:

```
System.out.println(  
    operation.apply(10, 20)  
);
```

Output:

Result = 30

Notice the type transformation:

```
(Integer,Integer)  
  ↓  
Integer  
  ↓  
String
```

So:

```
BiFunction<Integer,Integer,Integer>  
      +  
Function<Integer,String>  
      ↓  
BiFunction<Integer,Integer,String>
```

13. andThen() type parameters

The signature is conceptually:

```
<V> BiFunction<T,U,V> andThen(  
    Function<? super R, ? extends V> after)
```

The important pieces are:

T = first input
U = second input
R = current result
V = final result

Suppose:

```
BiFunction<Integer, Integer, Integer> add
```

and:

```
Function<Integer, String> format
```

Then:

T = Integer
U = Integer
R = Integer
V = String

Therefore:

```
BiFunction<Integer,Integer,String>
```

is produced.

14. andThen() versus Function.andThen()

Both interfaces have andThen(), but their input structure differs.

Function

$T \rightarrow R$

Then:

$R \rightarrow V$

Overall:

$T \rightarrow R \rightarrow V$

BiFunction

$(T,U) \rightarrow R$

Then:

$R \rightarrow V$

Overall:

$(T,U) \rightarrow R \rightarrow V$

So:

Function.andThen()
one input initially

BiFunction.andThen()
two inputs initially

15. BiFunction vs BinaryOperator

This is extremely important.

BiFunction<T,U,R>

$(T,U) \rightarrow R$

All three types can be different.

Example:

BiFunction<String, Integer, Double>

means:

String + Integer $\rightarrow$ Double

BinaryOperator<T>

$(T,T) \rightarrow T$

All three types must be the same.

Example:

BinaryOperator<Integer>

means:

Integer + Integer $\rightarrow$ Integer

Therefore:

BiFunction<T,U,R>

↓

general

BinaryOperator<T>

↓

specialized BiFunction<T,T,T>

16. BinaryOperator is a specialized BiFunction

This relationship is important:

BiFunction<T,U,R>

▲

BiFunction<T,T,T>

▲

BinaryOperator<T>

Conceptually:

BinaryOperator<Integer> add =
 (a, b) -> a + b;

is a specialized form of:

BiFunction<Integer, Integer, Integer>

17. BiFunction vs Function

Function

One input:

T $\rightarrow$ R

Example:

```
Function<String, Integer> length =  
    s -> s.length();
```

BiFunction

Two inputs:

$(T, U) \rightarrow R$

Example:

```
BiFunction<String, String, String> concat =  
    (a, b) -> a + b;
```

Therefore:

Function:
one input $\rightarrow$ one output

BiFunction:
two inputs $\rightarrow$ one output

18. BiFunction vs Predicate

Predicate

$T \rightarrow \text{boolean}$

Example:

```
Predicate<Integer> positive =  
    x -> x > 0;
```

BiFunction

$(T, U) \rightarrow R$

Example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

So:

Predicate:

one input $\rightarrow$ boolean

BiFunction:
two inputs $\rightarrow$ result

19. BiFunction vs Consumer

Consumer

$T \rightarrow \text{void}$

Example:

```
Consumer<String> print =  
    s -> System.out.println(s);
```

BiFunction

$(T,U) \rightarrow R$

Example:

```
BiFunction<Integer,Integer,Integer> add =  
    (a,b) -> a+b;
```

Therefore:

Consumer:
one input $\rightarrow$ action

BiFunction:
two inputs $\rightarrow$ result

20. BiFunction vs Supplier

Supplier

$() \rightarrow T$

No inputs.

BiFunction

$(T,U) \rightarrow R$

Two inputs.

So:

Supplier:
nothing $\rightarrow$ result

BiFunction:
two values $\rightarrow$ result

21. Real-world Employee example

Suppose:

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

Create a BiFunction that calculates total salary:

```
BiFunction<Employee, Employee, Double> totalSalary =  
    (e1, e2) ->  
        e1.salary + e2.salary;
```

Then:

```
Employee e1 =  
    new Employee("Amit", 50000);
```

```
Employee e2 =  
    new Employee("Rahul", 70000);
```

```
double result =  
    totalSalary.apply(e1, e2);
```

```
System.out.println(result);
```

Output:

120000.0

Type flow:

Employee + Employee



BiFunction



Double

22. Employee example with andThen()

Suppose we first calculate total salary:

```
BiFunction<Employee, Employee, Double> totalSalary =  
    (e1, e2) ->  
        e1.salary + e2.salary;
```

Then we want to format the result:

```
Function<Double, String> format =  
    salary ->  
        "Total Salary = " + salary;
```

Combine:

```
BiFunction<Employee, Employee, String> operation =  
    totalSalary.andThen(format);
```

Then:

```
System.out.println(  
    operation.apply(e1, e2)  
);
```

Output:

Total Salary = 120000.0

Execution:

```
Employee 1 ┌───> totalSalary ──> 120000.0  
Employee 2 └───  
           ↓  
         format  
           ↓  
    "Total Salary = 120000.0"
```

23. BiFunction with different input types

This is a very common use.

```
BiFunction<String, Integer, String> createEmployeeId =  
    (department, number) ->  
        department + "-" + number;
```

Call:

```
String id =  
    createEmployeeId.apply("IT", 101);
```

```
System.out.println(id);
```

Output:

IT-101

Here:

```
T = String  
U = Integer  
R = String
```

So:

$(\text{String}, \text{Integer}) \rightarrow \text{String}$

24. BiFunction with database-style logic

Suppose:

```
BiFunction<String, Double, String> createSalaryMessage =  
    (name, salary) ->  
        name + " earns " + salary;
```

Call:

```
System.out.println(  
    createSalaryMessage.apply(  
        "Anurag",  
        75000.0  
    )  
);
```

Output:

Anurag earns 75000.0

This demonstrates how BiFunction can combine two pieces of information into one result.

25. BiFunction with Map.compute()

BiFunction appears in several Java collection APIs.

For example:

```
Map<String, Integer> scores =  
    new HashMap<>();
```

```
scores.put("Java", 10);
```

Map.compute() accepts a remapping function whose shape is essentially:

$$(K,V) \rightarrow V$$

which matches:

BiFunction<K,V,V>

Example:

```
scores.compute(  
    "Java",  
    (key, value) -> value + 5  
);
```

Now:

Java → 15

The lambda:

(key, value) -> value + 5

is a BiFunction-style operation.

26. Map.merge() and BiFunction

Another excellent example is:

```
Map<String, Integer> counts =  
    new HashMap<>();
```

Suppose:

```
counts.put("Java", 10);
```

Then:

```
counts.merge(  
    "Java",  
    5,  
    (oldValue, newValue) ->  
        oldValue + newValue  
);
```

The third argument is essentially:

$(\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$

which is a:

`BiFunction<Integer,Integer,Integer>`

The calculation:

$10 + 5 = 15$

gives:

$\text{Java} \rightarrow 15$

27. BiFunction and method references

Suppose:

```
public static Integer add(  
    Integer a,  
    Integer b) {  
  
    return a + b;  
}
```

You can write:

```
BiFunction<Integer, Integer, Integer> operation =  
    BiFunctionDemo::add;
```

instead of:

```
BiFunction<Integer, Integer, Integer> operation =  
    (a, b) -> add(a, b);
```

Then:

```
System.out.println(  
    operation.apply(10, 20)  
);
```

Output:

30

The method reference:

BiFunctionDemo::add

matches:

$(\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$

28. BiFunction and instance method reference

Suppose:

```
class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

You can create:

```
Calculator calculator =  
    new Calculator();
```

```
BiFunction<Integer, Integer, Integer> add =  
    calculator::add;
```

Then:

```
System.out.println(  
    add.apply(10, 20)  
);
```

Output:

30

The method reference represents:

`Integer + Integer → Integer`

29. BiFunction and lambda equivalence

Lambda:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

Anonymous class conceptually:

```
BiFunction<Integer, Integer, Integer> add =  
    new BiFunction<Integer, Integer, Integer>() {  
  
        @Override  
        public Integer apply(  
            Integer a,  
            Integer b) {  
  
            return a + b;  
        }  
    };
```

Then:

```
add.apply(10, 20);
```

invokes the implementation.

30. andThen() with multiple stages

You can chain multiple operations.

Suppose:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

```
Function<Integer, Integer> multiply =
```

```
x -> x * 2;
```

```
Function<Integer, String> format =  
    x -> "Result = " + x;
```

Then:

```
BiFunction<Integer, Integer, String> operation =  
    add  
    .andThen(multiply)  
    .andThen(format);
```

Call:

```
System.out.println(  
    operation.apply(10, 20)  
);
```

Execution:

```
10 + 20  
↓  
30  
↓  
30 × 2  
↓  
60  
↓  
"Result = 60"
```

Output:

Result = 60

31. The complete andThen() algorithm

For:

```
add  
    .andThen(multiply)  
    .andThen(format)
```

the conceptual algorithm is:

Step 1:

Receive a and b

↓

Step 2:

```
result1 = add.apply(a,b)
```

↓

Step 3:

```
result2 = multiply.apply(result1)
```

↓

Step 4:

```
result3 = format.apply(result2)
```

↓

Return result3

So:

(T,U)

↓

BiFunction

↓

R

↓

Function

↓

V

↓

Function

↓

W

32. Exception behavior

If the first BiFunction throws an exception:

```
BiFunction<Integer, Integer, Integer> first =  
    (a, b) -> {  
        throw new RuntimeException();  
    };
```

```
Function<Integer, String> second =  
    x -> "Result = " + x;
```

Then:

```
BiFunction<Integer, Integer, String> combined =  
    first.andThen(second);
```

Calling:

```
combined.apply(10, 20);
```

results in:

```
first  
↓  
exception  
↓  
second is NOT executed
```

The second Function is executed only if the first operation completes successfully.

33. andThen(null)

The after function cannot be null.

For example:

```
Function<Integer, String> after = null;
```

```
BiFunction<Integer, Integer, String> combined =  
    add.andThen(after);
```

This results in a NullPointerException.

So:

```
andThen()  
↓  
requires non-null Function
```

34. BiFunction and functional-interface rules

Why is:

```
BiFunction<T,U,R>
```

a functional interface even though it has andThen()?

Because:

`apply()`

is the only abstract method.

Whereas:

`andThen()`

is already implemented as a default method.

Therefore:

`BiFunction`

```
|  
├── apply() → abstract  
└── andThen() → default
```

A lambda only needs to implement:

`R apply(T t, U u);`

35. Important comparison of Java 8 interfaces

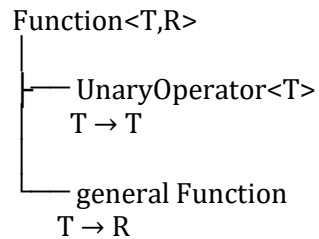
You have now covered the major interfaces. They can be summarized as:

Interface	Inputs	Output	Main abstract method	Default methods
Function<T,R>	1	1	<code>apply()</code>	<code>compose()</code> , <code>andThen()</code>
BiFunction<T,U,R>	2	1	<code>apply()</code>	<code>andThen()</code>
UnaryOperator<T>	1	1	<code>apply()</code>	inherited <code>compose()</code> , <code>andThen()</code>
BinaryOperator<T>	2	1	<code>apply()</code>	inherited <code>andThen()</code>
Predicate<T>	1	boolean	<code>test()</code>	<code>and()</code> , <code>or()</code> , <code>negate()</code>
Consumer<T>	1	void	<code>accept()</code>	<code>andThen()</code>
Supplier<T>	0	1	<code>get()</code>	none

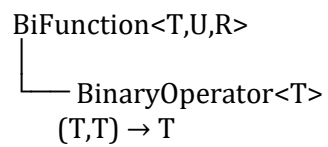
The critical distinction is the **number of inputs and type relationship**.

36. Function family hierarchy

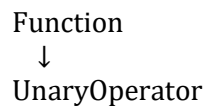
A useful way to remember the relationships is:



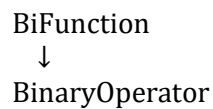
and:



So:



and:



The specialized operators simply enforce that the input/output types are the same.

37. Complete practical example

```
import java.util.function.BiFunction;
import java.util.function.Function;

public class BiFunctionDemo {

    public static void main(String[] args) {

        // BiFunction: two inputs → one output
        BiFunction<Integer, Integer, Integer> add =
            (a, b) -> a + b;
```

```

// apply()
int sum =
    add.apply(10, 20);

System.out.println(
    "Sum = " + sum
);

// Function executed after BiFunction
Function<Integer, Integer> doubleValue =
    x -> x * 2;

BiFunction<Integer, Integer, Integer> combined =
    add.andThen(doubleValue);

int result =
    combined.apply(10, 20);

System.out.println(
    "Result = " + result
);

// Different final output type
Function<Integer, String> format =
    x -> "Final Result = " + x;

BiFunction<Integer, Integer, String> formatted =
    add.andThen(format);

System.out.println(
    formatted.apply(10, 20)
);
}
}

```

Output:

```

Sum = 30
Result = 60
Final Result = 30

```

38. Internal execution model

Consider:

```

BiFunction<Integer, Integer, Integer> add =
    (a, b) -> a + b;

```

When:

```
add.apply(10, 20);
```

is executed:

```
1. Call apply(10,20)
  ↓
2. a = 10
3. b = 20
  ↓
4. Execute lambda body
  ↓
5. a + b
  ↓
6. 10 + 20
  ↓
7. Return 30
```

For:

```
BiFunction<Integer, Integer, String> formatted =
    add.andThen(format);
```

the algorithm becomes:

```
10,20
  ↓
add.apply()
  ↓
30
  ↓
format.apply()
  ↓
"Final Result = 30"
```

39. Interview-ready answer

If an interviewer asks:

What is BiFunction<T,U,R>?

BiFunction<T,U,R> is a Java 8 functional interface that accepts two arguments of types T and U and returns a result of type R.

Its functional shape is:

$(T,U) \rightarrow R$

What is its abstract method?

`R apply(T t, U u);`

Is `apply()` a default method?

No. `apply()` is the **abstract functional method**.

What is `andThen()`?

`andThen()` is a **default method** that takes a Function and applies it to the result produced by the BiFunction.

Conceptually:

$(T, U) \rightarrow R \rightarrow V$

Example:

```
BiFunction<Integer,Integer,Integer> add =  
    (a,b) -> a+b;
```

```
Function<Integer,String> format =  
    x -> "Result = " + x;
```

```
BiFunction<Integer,Integer,String> combined =  
    add.andThen(format);
```

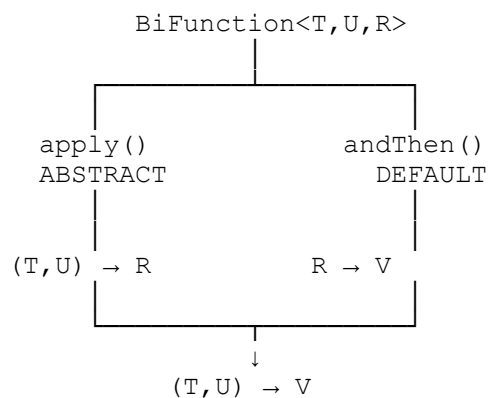
Then:

```
combined.apply(10,20);
```

produces:

Result = 30

40. Final mental model



The easiest mnemonic is:

BiFunction = "Give me TWO values, and I'll give you ONE result."

For example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

means:

```
Integer  
  +  
Integer  
  ↓  
Integer
```

And:

```
add.andThen(format)
```

means:

```
Integer + Integer  
  ↓  
  add  
  ↓  
Integer  
  ↓  
format  
  ↓  
String
```

So the technically correct classification is:

```
BiFunction<T,U,R>  
├── apply(T,U) → ABSTRACT  
└── andThen() → DEFAULT
```

The key difference from BinaryOperator<T> is:

```
BiFunction<T,U,R> → (T,U) → R  
BinaryOperator<T> → (T,T) → T
```

In other words, **BinaryOperator<T> is essentially a specialized BiFunction<T,T,T>.**

Yes. One important correction first:

BiPredicate<T,U> is a functional interface whose abstract method is test(T,U). test() is not a default method. and(), or(), and negate() are default methods.

Its technically correct structure is:

```
BiPredicate<T,U>
├── test(T,U) → abstract
├── and()      → default
├── or()       → default
└── negate()  → default
```

Its functional shape is:

$(T, U) \rightarrow \text{boolean}$

1. What is BiPredicate<T,U>?

BiPredicate<T,U> is a built-in Java functional interface in:

java.util.function.BiPredicate

It represents a condition or test that:

- accepts **two input values**
- evaluates some condition
- returns true or false

The fundamental form is:

$(T, U) \rightarrow \text{boolean}$

For example:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Calling:

```
greater.test(20, 10);
```

produces:

true

Because:

$20 > 10$

2. Why is it called BiPredicate?

The word **Bi** means two.

A normal:

`Predicate<T>`

takes one input:

$T \rightarrow \text{boolean}$

A:

`BiPredicate<T,U>`

takes two inputs:

$(T,U) \rightarrow \text{boolean}$

Comparison:

`Predicate:`
 T
 ↓
 condition
 ↓
 true/false

`BiPredicate:`
 T  $\rightarrow$ condition $\rightarrow$ true/false
 U

3. Structure of BiPredicate

Conceptually, Java's interface is:

```
@FunctionalInterface
public interface BiPredicate<T, U> {

    boolean test(T t, U u);
}
```

```

default BiPredicate<T, U> and(
    BiPredicate<? super T, ? super U> other) {
    ...
}

default BiPredicate<T, U> or(
    BiPredicate<? super T, ? super U> other) {
    ...
}

default BiPredicate<T, U> negate() {
    ...
}
}

```

Therefore:

Method	Type	Purpose
test(T,U)	Abstract	Perform the two-argument test
and()	Default	Logical AND
or()	Default	Logical OR
negate()	Default	Reverse the result

Unlike some other functional interfaces, BiPredicate does not have a static method such as identity().

4. test() — the abstract method

The main functional method is:

```
boolean test(T t, U u);
```

It accepts two arguments and returns:

true

or:

false

Example:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

```
System.out.println(  
    greater.test(20, 10)  
);
```

Output:

true

Execution:

```
20  
↓  
test(20,10)  
↑  
10  
↓  
20 > 10  
↓  
true
```

5. Why is test() abstract?

Because Java does not know what condition you want to test.

For example:

Greater than

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Equal

```
BiPredicate<Integer, Integer> equal =  
    (a, b) -> a.equals(b);
```

Divisible

```
BiPredicate<Integer, Integer> divisible =  
    (a, b) -> a % b == 0;
```

String starts with another string

```
BiPredicate<String, String> startsWith =
```

```
(text, prefix) -> text.startsWith(prefix);
```

All have the same functional shape:

$(T, U) \rightarrow \text{boolean}$

but perform different tests.

6. Basic example

```
import java.util.function.BiPredicate;

public class BiPredicateDemo {

    public static void main(String[] args) {

        BiPredicate<Integer, Integer> greater =
            (a, b) -> a > b;

        boolean result =
            greater.test(50, 20);

        System.out.println(result);
    }
}
```

Output:

true

Algorithm:

1. Receive a = 50
 2. Receive b = 20
 3. Evaluate a > b
 4. Evaluate 50 > 20
 5. Return true
-

7. BiPredicate can use different input types

Unlike BinaryOperator, the two input types do not have to be the same.

For example:

```
BiPredicate<String, Integer> lengthGreaterThan =
    (text, length) ->
```

```
text.length() > length;
```

Then:

```
System.out.println(  
    lengthGreaterThan.test("Java Programming", 10)  
);
```

Output:

true

Here:

```
T = String  
U = Integer
```

So the shape is:

$(\text{String}, \text{Integer}) \rightarrow \text{boolean}$

This is a major feature of BiPredicate.

8. BiPredicate vs Predicate

This distinction is very important.

Predicate

```
Predicate<Integer> positive =  
    x -> x > 0;
```

Shape:

$\text{Integer} \rightarrow \text{boolean}$

BiPredicate

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Shape:

$(\text{Integer}, \text{Integer}) \rightarrow \text{boolean}$

Therefore:

Predicate<T>



one input



boolean

BiPredicate<T,U>



two inputs



boolean

9. and() — default method

and() is a **default method**.

Its purpose is to combine two BiPredicate conditions using logical AND.

Suppose:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

```
BiPredicate<Integer, Integer> even =  
    (a, b) -> a % 2 == 0;
```

Combine them:

```
BiPredicate<Integer, Integer> combined =  
    greater.and(even);
```

Now:

```
combined.test(20, 10);
```

evaluates:

```
20 > 10
```

```
AND
```

```
20 % 2 == 0
```

Both are true:

```
true AND true = true
```

Therefore:

true

10. and() algorithm

Consider:

```
BiPredicate<Integer, Integer> p1 =  
    (a, b) -> a > b;
```

```
BiPredicate<Integer, Integer> p2 =  
    (a, b) -> a % 2 == 0;
```

```
BiPredicate<Integer, Integer> combined =  
    p1.and(p2);
```

Calling:

```
combined.test(20, 10);
```

conceptually executes:

```
p1.test(20,10)
```

↓

```
true
```

↓

```
p2.test(20,10)
```

↓

```
true
```

↓

```
true AND true
```

↓

```
true
```

11. and() short-circuit behavior

This is very important.

and() uses **short-circuit logical AND**.

Suppose:

```
BiPredicate<Integer, Integer> first =  
    (a, b) -> false;
```

```
BiPredicate<Integer, Integer> second =  
    (a, b) -> {  
        System.out.println("Second executed");
```

```
        return true;
    };
```

Combine:

```
BiPredicate<Integer, Integer> combined =
    first.and(second);
```

Now:

```
combined.test(10, 20);
```

Output:

false

It does **not** print:

Second executed

because the first predicate is already false.

Conceptually:

```
false AND ?
  ↓
false
```

There is no need to evaluate the second condition.

12. or() — default method

or() combines two BiPredicate conditions using logical OR.

Example:

```
BiPredicate<Integer, Integer> greater =
    (a, b) -> a > b;
```

```
BiPredicate<Integer, Integer> equal =
    (a, b) -> a.equals(b);
```

Combine:

```
BiPredicate<Integer, Integer> greaterOrEqual =
    greater.or(equal);
```

Now:

```
greaterOrEqual.test(20, 20);
```

First condition:

$20 > 20$

is:

false

Second condition:

$20 == 20$

is:

true

Therefore:

false OR true = true

13. or() algorithm

Conceptually:

```
p1.test(a,b)
  ↓
false
  ↓
p2.test(a,b)
  ↓
true
  ↓
false OR true
  ↓
true
```

So:

```
true OR anything = true
false OR false = false
false OR true = true
```

14. or() also short-circuits

Consider:

```
BiPredicate<Integer, Integer> first =  
    (a, b) -> true;  
  
BiPredicate<Integer, Integer> second =  
    (a, b) -> {  
        System.out.println("Second executed");  
        return false;  
    };
```

Combine:

```
BiPredicate<Integer, Integer> combined =  
    first.or(second);
```

Calling:

```
combined.test(10, 20);
```

returns:

true

and the second predicate does not need to execute.

Because:

```
true OR ?  
  ↓  
true
```

15. negate() — default method

`negate()` reverses the result of a `BiPredicate`.

Suppose:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Then:

```
BiPredicate<Integer, Integer> notGreater =  
    greater.negate();
```

Now:

```
notGreater.test(10, 20);
```

Original condition:

$10 > 20$

is:

false

Negated:

!false

becomes:

true

16. negate() algorithm

Conceptually:

```
original.test(a,b)
  ↓
  result
  ↓
 NOT result
  ↓
final result
```

For example:

```
greater.test(10,20)
  ↓
  false
  ↓
negate()
  ↓
  true
```

17. Complete example of and(), or(), and negate()

```
import java.util.function.BiPredicate;
```

```

public class BiPredicateDemo {

    public static void main(String[] args) {

        BiPredicate<Integer, Integer> greater =
            (a, b) -> a > b;

        BiPredicate<Integer, Integer> even =
            (a, b) -> a % 2 == 0;

        // and()
        BiPredicate<Integer, Integer> greaterAndEven =
            greater.and(even);

        System.out.println(
            greaterAndEven.test(20, 10)
        );

        // or()
        BiPredicate<Integer, Integer> greaterOrEven =
            greater.or(even);

        System.out.println(
            greaterOrEven.test(10, 5)
        );

        // negate()
        BiPredicate<Integer, Integer> notGreater =
            greater.negate();

        System.out.println(
            notGreater.test(10, 20)
        );
    }
}

```

Output:

```

true
true
true

```

18. Combining all three methods

You can chain the default methods.

For example:

```
BiPredicate<Integer, Integer> p1 =  
    (a, b) -> a > b;
```

```
BiPredicate<Integer, Integer> p2 =  
    (a, b) -> a % 2 == 0;
```

```
BiPredicate<Integer, Integer> p3 =  
    (a, b) -> b % 2 == 0;
```

You could write:

```
BiPredicate<Integer, Integer> combined =  
    p1.and(p2).or(p3);
```

The logical structure is:

(p1 AND p2) OR p3

For:

```
combined.test(20, 10);
```

we get:

```
p1:  
20 > 10  
true
```

```
p2:  
20 is even  
true
```

```
p1 AND p2:  
true
```

```
p3:  
10 is even  
true
```

```
true OR true:  
true
```

19. Parentheses and chaining

This is important when creating complex predicates.

Consider:

$p1.and(p2).or(p3)$

Conceptually:

$(p1 \text{ AND } p2) \text{ OR } p3$

Whereas:

$p1.and(p2.or(p3))$

means:

$p1 \text{ AND } (p2 \text{ OR } p3)$

These are logically different.

For complex business rules, build predicates carefully so the intended grouping is obvious.

20. Real-world Employee example

Suppose:

```
class Employee {  
  
    String name;  
    double salary;  
    int age;  
  
    Employee(  
        String name,  
        double salary,  
        int age) {  
  
        this.name = name;  
        this.salary = salary;  
        this.age = age;  
    }  
}
```

We can create a BiPredicate that compares two employees.

For example:

```
BiPredicate<Employee, Employee> higherSalary =  
    (e1, e2) ->  
        e1.salary > e2.salary;
```

Then:

```
Employee e1 =  
    new Employee("Amit", 50000, 30);  
  
Employee e2 =  
    new Employee("Rahul", 70000, 35);  
  
System.out.println(  
    higherSalary.test(e1, e2)  
);
```

Output:

true

Because:

70000 > 50000

21. BiPredicate comparing two Employees

You can create multiple conditions.

```
BiPredicate<Employee, Employee> salaryGreater =  
    (e1, e2) ->  
        e1.salary > e2.salary;
```

```
BiPredicate<Employee, Employee> ageGreater =  
    (e1, e2) ->  
        e1.age > e2.age;
```

Combine:

```
BiPredicate<Employee, Employee> salaryAndAgeGreater =  
    salaryGreater.and(ageGreater);
```

Then:

```
salaryAndAgeGreater.test(e1, e2);
```

The condition is:

```
e1.salary > e2.salary  
AND  
e1.age > e2.age
```

22. Practical validation example

Suppose you need to validate a username and password together.

```
BiPredicate<String, String> validCredentials =  
    (username, password) ->  
        username != null  
        && password != null  
        && !username.isEmpty()  
        && password.length() >= 8;
```

Then:

```
boolean valid =  
    validCredentials.test(  
        "anurag",  
        "password123"  
    );
```

Result:

true

The two inputs are:

```
String username  
String password
```

and the output is:

boolean

Therefore:

$(\text{String}, \text{String}) \rightarrow \text{boolean}$

23. BiPredicate with different types

For example, validate whether a string has a required minimum length:

```
BiPredicate<String, Integer> minimumLength =  
    (text, length) ->  
        text != null &&  
        text.length() >= length;
```

Call:

```
System.out.println(
    minimumLength.test("Java Programming", 10)
);
```

Output:

true

Here:

String + Integer → boolean

This demonstrates why BiPredicate<T,U> is more general than a predicate that accepts two values of the same type.

24. BiPredicate with Map

BiPredicate can be useful when you need to validate a key-value pair.

For example:

```
BiPredicate<String, Integer> validScore =
    (subject, score) ->
        subject != null &&
        score >= 0 &&
        score <= 100;
```

Then:

```
validScore.test("Java", 85);
```

returns:

true

Conceptually:

```
key/value
↓
BiPredicate
↓
valid / invalid
```

25. Method references with BiPredicate

Suppose you have:

```
public static boolean isGreater(  
    Integer a,  
    Integer b) {  
  
    return a > b;  
}
```

You can write:

```
BiPredicate<Integer, Integer> greater =  
    BiPredicateDemo::isGreater;
```

instead of:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> isGreater(a, b);
```

Then:

```
greater.test(20, 10);
```

returns:

true

The method reference matches:

$(\text{Integer}, \text{Integer}) \rightarrow \text{boolean}$

26. Anonymous class equivalent

Lambda:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Conceptually corresponds to:

```
BiPredicate<Integer, Integer> greater =  
    new BiPredicate<Integer, Integer>() {  
  
        @Override  
        public boolean test(  
            Integer a,  
            Integer b) {
```

```
        return a > b;
    }
};
```

The lambda provides the implementation of the one abstract method:

```
test()
```

27. Why is BiPredicate a functional interface?

Because it has exactly **one abstract method**:

```
boolean test(T t, U u);
```

The other methods are default methods:

```
and()
or()
negate()
```

Therefore:

```
BiPredicate<Integer, Integer> greater =
    (a, b) -> a > b;
```

is valid.

The lambda implements:

```
test(a,b)
```

28. Internal logic of and()

The default and() method is conceptually similar to:

```
default BiPredicate<T, U> and(
    BiPredicate<? super T, ? super U> other) {

    return (t, u) ->
        test(t, u) &&
        other.test(t, u);
}
```

The important part is:

`test(t, u) && other.test(t, u)`

Therefore it uses Java's short-circuit `&&`.

Execution:

```
first.test()
  ↓
false?
 /    \
yes     no
 ↓      ↓
return  execute second
false
```

29. Internal logic of `or()`

Conceptually:

```
default BiPredicate<T, U> or(
    BiPredicate<? super T, ? super U> other) {

    return (t, u) ->
        test(t, u) ||
        other.test(t, u);
}
```

The important part is:

`test(t,u) || other.test(t,u)`

Therefore it uses Java's short-circuit `||`.

Execution:

```
first.test()
  ↓
true?
 /    \
yes     no
 ↓      ↓
return  execute second
true
```

30. Internal logic of negate()

Conceptually:

```
default BiPredicate<T, U> negate() {  
    return (t, u) ->  
        !test(t, u);  
}
```

So:

`predicate.negate()`

is essentially:

`(t, u) -> !predicate.test(t, u)`

For example:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

Then:

`greater.negate()`

is conceptually:

`(a, b) -> !(a > b);`

31. BiPredicate vs BiFunction

This is another key distinction.

BiFunction

$(T, U) \rightarrow R$

Returns a value.

Example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;  
10, 20 → 30
```

BiPredicate

$(T,U) \rightarrow \text{boolean}$

Returns only a condition.

Example:

```
BiPredicate<Integer, Integer> greater =  
    (a,b) -> a>b;  
10,20 → false
```

So:

BiFunction = calculate / transform
BiPredicate = test / validate

32. BiPredicate vs BinaryOperator

BinaryOperator

$(T,T) \rightarrow T$

Example:

```
BinaryOperator<Integer> add =  
    (a,b) -> a+b;
```

Result:

$10 + 20 \rightarrow 30$

BiPredicate

$(T,U) \rightarrow \text{boolean}$

Example:

```
BiPredicate<Integer,Integer> greater =  
    (a,b) -> a>b;
```

Result:

$10,20 \rightarrow \text{false}$

So:

BinaryOperator = produce a value
BiPredicate = produce true/false

33. BiPredicate vs BiConsumer

There is no standard BiConsumer composition exactly analogous to BiPredicate's logical methods.

BiConsumer<T,U> has:

void accept(T t, U u);

while:

BiPredicate<T,U>

has:

boolean test(T t, U u);

Therefore:

BiConsumer:
(T,U) → void

BiPredicate:
(T,U) → boolean

34. Complete example: employee validation

```
import java.util.function.BiPredicate;
```

```
class Employee {
```

```
    String name;  
    double salary;
```

```
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

```
public class BiPredicateEmployeeDemo {
```

```
    public static void main(String[] args) {
```

```
        Employee e1 =
```

```

        new Employee("Amit", 60000);

Employee e2 =
    new Employee("Rahul", 50000);

BiPredicate<Employee, Employee> salaryGreater =
    (a, b) ->
        a.salary > b.salary;

System.out.println(
    salaryGreater.test(e1, e2)
);

BiPredicate<Employee, Employee> salaryEqual =
    (a, b) ->
        a.salary == b.salary;

BiPredicate<Employee, Employee> greaterOrEqual =
    salaryGreater.or(salaryEqual);

System.out.println(
    greaterOrEqual.test(e1, e2)
);

BiPredicate<Employee, Employee> notGreater =
    salaryGreater.negate();

System.out.println(
    notGreater.test(e1, e2)
);
    }
}

```

Output:

```

true
true
false

```

35. Method classification

This is the most important table to remember:

Method	Type	Purpose
test(T,U)	Abstract	Evaluate condition
and(...)	Default	Logical AND

or(...)	Default	Logical OR
negate()	Default	Reverse result

So the correct version of your statement is:

BiPredicate<T,U> has the abstract test(T,U) method and default and(), or(), and negate() methods.

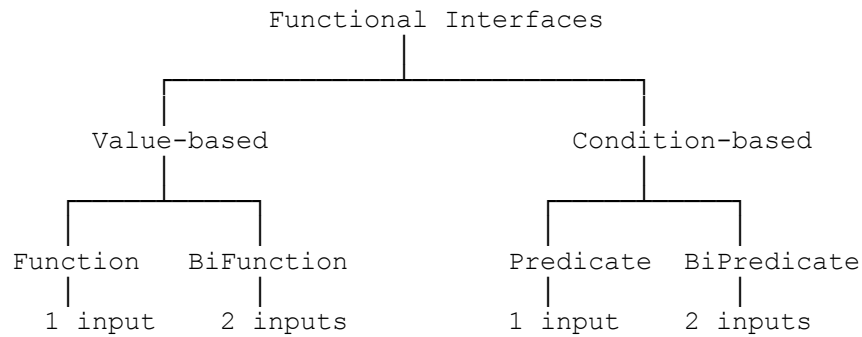
test() is **not** a default method.

36. Comparison with the other Java 8 functional interfaces

You have now covered several of the most important interfaces:

Interface	Input	Output	Abstract method	Default methods
Function<T,R>	1	R	apply()	compose(), andThen()
BiFunction<T,U,R>	2	R	apply()	andThen()
Predicate<T>	1	boolean	test()	and(), or(), negate()
BiPredicate<T,U>	2	boolean	test()	and(), or(), negate()
Consumer<T>	1	void	accept()	andThen()
Supplier<T>	0	T	get()	none
UnaryOperator<T>	1	T	apply()	inherited compose(), andThen()
BinaryOperator<T>	2	T	apply()	inherited andThen()

This gives you a very useful pattern:



37. The most important mental model

Remember these four:

`Function<T,R>`
 $T \rightarrow R$

`BiFunction<T,U,R>`
 $(T,U) \rightarrow R$

`Predicate<T>`
 $T \rightarrow \text{boolean}$

`BiPredicate<T,U>`
 $(T,U) \rightarrow \text{boolean}$

So the difference between `Predicate` and `BiPredicate` is simply the number of inputs:

`Predicate:`

T
 $\downarrow$
test
 $\downarrow$
boolean

`BiPredicate:`

T $\downarrow$
 U $\downarrow$
test $\rightarrow$ boolean

38. Interview-ready answer

If asked:

What is BiPredicate?

BiPredicate<T,U> is a Java 8 functional interface that represents a boolean-valued condition involving two input arguments.

Its functional shape is:

$(T,U) \rightarrow \text{boolean}$

What is its abstract method?

`boolean test(T t, U u);`

Is test() a default method?

No. It is the single **abstract method**.

What are its default methods?

`and()`
`or()`
`negate()`

What does and() do?

Combines two BiPredicates using logical AND:

`p1.and(p2)`

Conceptually:

$(a,b) \rightarrow p1.test(a,b) \ \&\& \ p2.test(a,b)$

What does or() do?

Combines them using logical OR:

`p1.or(p2)`

Conceptually:

$(a,b) \rightarrow p1.test(a,b) \ || \ p2.test(a,b)$

What does negate() do?

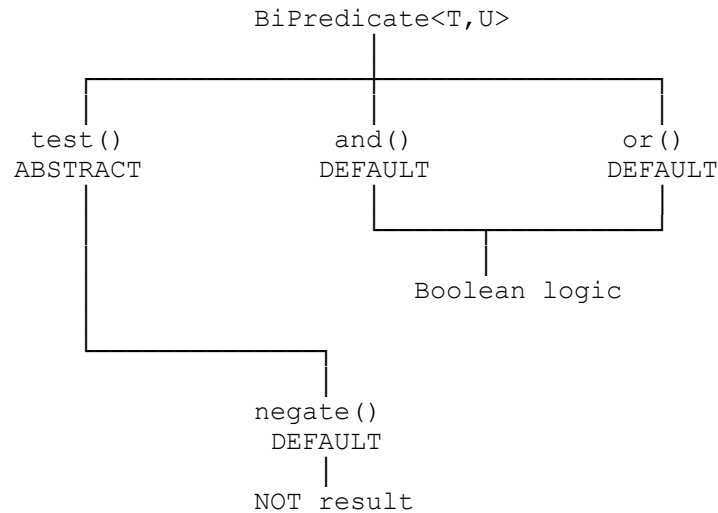
Reverses the result:

`p.negate()`

Conceptually:

$(a,b) \rightarrow !p.test(a,b)$

39. Final mental model



The easiest mnemonic is:

BiPredicate = "Give me TWO values; I'll tell you TRUE or FALSE."

For example:

```
BiPredicate<Integer, Integer> greater =
    (a, b) -> a > b;
```

means:

```
Integer ┌──┐
         │  │ test() ──> boolean
Integer └──┘
```

And the three composition methods are:

and() → both conditions must be true

or() → at least one condition must be true

negate() → reverse true ↔ false

So the technically correct classification is:

```
BiPredicate<T,U>
└─ test(T,U) → ABSTRACT
```

```
|— and()   → DEFAULT
|— or()    → DEFAULT
|— negate() → DEFAULT
```

Key point: test() is the functional/abstract method; and(), or(), and negate() are the built-in default methods that allow BiPredicates to be composed into more complex boolean conditions.

Yes. There is one important correction to your statement:

BiConsumer<T,U> is a functional interface whose abstract method is accept(T,U). accept() is not a default method. andThen() is the default method.

The technically correct structure is:

```
BiConsumer<T,U>
|
|— accept(T,U) → abstract
|— andThen()   → default
```

Its functional shape is:

(T, U) → void

1. What is BiConsumer<T,U>?

BiConsumer<T,U> is a built-in functional interface introduced in Java 8 and available in:

java.util.function.BiConsumer

It represents an operation that:

- accepts **two input values**
- performs some action
- **does not return a result**

Its fundamental form is:

(T, U) → void

For example:

```
BiConsumer<String, Integer> printEmployee =
    (name, age) ->
        System.out.println(
            name + " : " + age
```

);

Calling:

```
printEmployee.accept("Anurag", 30);
```

produces an action/output, but the BiConsumer itself returns nothing.

2. Why is it called BiConsumer?

The word **Bi** means two.

A normal:

`Consumer<T>`

accepts one value:

$T \rightarrow \text{void}$

A:

`BiConsumer<T,U>`

accepts two values:

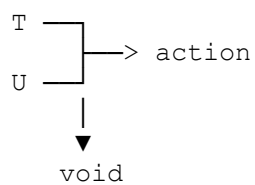
$(T,U) \rightarrow \text{void}$

Comparison:

Consumer:



BiConsumer:



3. Structure of BiConsumer

Conceptually, Java's interface looks like:

```
@FunctionalInterface
public interface BiConsumer<T, U> {

    void accept(T t, U u);

    default BiConsumer<T, U> andThen(
        BiConsumer<? super T, ? super U> after) {
        ...
    }
}
```

Therefore:

Method	Type	Purpose
accept(T,U)	Abstract	Perform the operation
andThen()	Default	Execute another BiConsumer afterward

Unlike BiFunction, BiConsumer has no return value.

4. accept() — the abstract method

The main functional method is:

```
void accept(T t, U u);
```

It:

- receives two arguments
- performs an action
- returns nothing

Example:

```
BiConsumer<String, Integer> print =
    (name, age) ->
        System.out.println(
            name + " : " + age
        );
```

```
print.accept("Anurag", 30);
```

Output:

Anurag : 30

Notice:

```
print.accept(...)
```

does not produce a value that can be assigned to a variable.

This would be invalid:

```
int result = print.accept("Anurag", 30);
```

because:

```
accept() → void
```

5. Basic BiConsumer example

```
import java.util.function.BiConsumer;
```

```
public class BiConsumerDemo {  
    public static void main(String[] args) {  
        BiConsumer<String, Integer> print =  
            (name, age) ->  
                System.out.println(  
                    name + " : " + age  
                );  
        print.accept("Anurag", 30);  
    }  
}
```

Output:

Anurag : 30

Execution algorithm:

1. Receive first argument = "Anurag"
2. Receive second argument = 30

3. Execute lambda body
 4. Print the values
 5. Return nothing
-

6. BiConsumer can accept different types

One of the important characteristics of BiConsumer is that T and U can be different types.

For example:

```
BiConsumer<String, Integer> display =  
    (text, number) ->  
        System.out.println(  
            text + " = " + number  
        );
```

Here:

T = String
U = Integer

So the functional shape is:

(String, Integer) → void

Call:

```
display.accept("Age", 30);
```

Output:

Age = 30

7. accept() is NOT a default method

This is the most important correction.

Your statement says:

"BiConsumer functional interfaces include default methods such as accept(), andThen()."

The technically correct statement is:

BiConsumer<T,U> has one abstract method, accept(T,U), and one default method, andThen().

Why?

Because a functional interface must have exactly one abstract method.

```
BiConsumer<T,U>
├── accept() → ABSTRACT
└── andThen() → DEFAULT
```

The lambda expression supplies the implementation of accept().

8. andThen() — the default method

andThen() allows you to execute another BiConsumer **after** the current BiConsumer.

Suppose:

```
BiConsumer<String, Integer> first =
    (name, age) ->
        System.out.println(
            "Name = " + name
        );
```

And:

```
BiConsumer<String, Integer> second =
    (name, age) ->
        System.out.println(
            "Age = " + age
        );
```

Combine them:

```
BiConsumer<String, Integer> combined =
    first.andThen(second);
```

Then:

```
combined.accept("Anurag", 30);
```

Output:

```
Name = Anurag
Age = 30
```

9. andThen() execution sequence

Consider:

```
BiConsumer<String, Integer> first =  
    (name, age) ->  
        System.out.println(  
            "Name = " + name  
        );
```

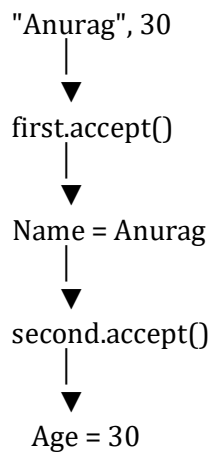
```
BiConsumer<String, Integer> second =  
    (name, age) ->  
        System.out.println(  
            "Age = " + age  
        );
```

```
BiConsumer<String, Integer> combined =  
    first.andThen(second);
```

When:

```
combined.accept("Anurag", 30);
```

is called:



So andThen() means:

```
first action  
↓  
second action
```

10. Conceptual implementation of andThen()

Conceptually, the default method behaves approximately like:

```

default BiConsumer<T, U> andThen(
    BiConsumer<? super T, ? super U> after) {

    Objects.requireNonNull(after);

    return (t, u) -> {
        accept(t, u);
        after.accept(t, u);
    };
}

```

The important part is:

```

accept(t, u);
after.accept(t, u);

```

Therefore:

```

first.accept()
  ↓
second.accept()

```

11. andThen() does not transform a value

This is an important difference between BiConsumer and BiFunction.

A BiFunction produces a result:

$(T, U) \rightarrow R$

so its andThen() can take that **result** and pass it to another Function.

A BiConsumer produces no result:

$(T, U) \rightarrow \text{void}$

Therefore its andThen() simply executes another BiConsumer using the **same two inputs**.

Compare:

BiFunction

```

(T,U)
  ↓
BiFunction
  ↓
R

```

↓
Function
↓
V

BiConsumer

(T,U)
↓
BiConsumer #1
↓
void
↓
BiConsumer #2
↓
void

12. BiConsumer vs BiFunction

This is one of the most important comparisons.

BiFunction

$(T,U) \rightarrow R$

Example:

```
BiFunction<Integer, Integer, Integer> add =  
    (a, b) -> a + b;
```

It returns:

30

BiConsumer

$(T,U) \rightarrow \text{void}$

Example:

```
BiConsumer<Integer, Integer> print =  
    (a, b) ->  
        System.out.println(a + b);
```

It performs an action but returns nothing.

So:

BiFunction = calculate/transform and RETURN
BiConsumer = perform action and RETURN NOTHING

13. BiConsumer vs Consumer

Consumer

$T \rightarrow \text{void}$

Example:

```
Consumer<String> print =  
    name -> System.out.println(name);
```

One input.

BiConsumer

$(T,U) \rightarrow \text{void}$

Example:

```
BiConsumer<String, Integer> print =  
    (name, age) ->  
        System.out.println(name + " " + age);
```

Two inputs.

Therefore:

Consumer:
one input $\rightarrow$ action

BiConsumer:
two inputs $\rightarrow$ action

14. BiConsumer vs BiPredicate

BiPredicate

$(T,U) \rightarrow \text{boolean}$

It asks a question.

Example:

```
BiPredicate<Integer, Integer> greater =  
    (a, b) -> a > b;
```

BiConsumer

$(T,U) \rightarrow \text{void}$

It performs an action.

Example:

```
BiConsumer<Integer, Integer> print =  
    (a, b) ->  
        System.out.println(a + b);
```

So:

```
BiPredicate = test  
BiConsumer = action
```

15. BiConsumer vs BinaryOperator

BinaryOperator

$(T,T) \rightarrow T$

Example:

```
BinaryOperator<Integer> add =  
    (a, b) -> a + b;
```

Returns:

30

BiConsumer

$(T,U) \rightarrow \text{void}$

Example:

```
BiConsumer<Integer, Integer> print =  
    (a,b) ->  
        System.out.println(a+b);
```

Prints:

but returns nothing.

16. Real-world Employee example

Suppose:

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

You can create a BiConsumer that updates two employees.

For example:

```
BiConsumer<Employee, Employee> printEmployees =  
    (e1, e2) -> {  
        System.out.println(  
            e1.name + " : " + e1.salary  
        );  
  
        System.out.println(  
            e2.name + " : " + e2.salary  
        );  
    };  
};
```

Then:

```
Employee e1 =  
    new Employee("Amit", 50000);
```

```
Employee e2 =  
    new Employee("Rahul", 70000);
```

```
printEmployees.accept(e1, e2);
```

Output:

Amit : 50000.0

Rahul : 70000.0

17. Employee example with andThen()

Suppose we want two actions:

Action 1: Print employee details

```
BiConsumer<Employee, Employee> print =  
    (e1, e2) -> {  
        System.out.println(  
            e1.name + " : " + e1.salary  
        );  
  
        System.out.println(  
            e2.name + " : " + e2.salary  
        );  
    };
```

Action 2: Print total salary

```
BiConsumer<Employee, Employee> total =  
    (e1, e2) -> {  
        System.out.println(  
            "Total Salary = "  
            + (e1.salary + e2.salary)  
        );  
    };
```

Combine:

```
BiConsumer<Employee, Employee> combined =  
    print.andThen(total);
```

Call:

```
combined.accept(e1, e2);
```

Output:

```
Amit : 50000.0  
Rahul : 70000.0  
Total Salary = 120000.0
```

Execution:

```
Employee 1 ┌───> print  
Employee 2 └───
```

↓
employee details
↓
total
↓
total salary

18. Chaining multiple BiConsumers

You can chain more than two.

```
BiConsumer<String, Integer> first =  
    (name, age) ->  
        System.out.println(  
            "Name: " + name  
        );  
  
BiConsumer<String, Integer> second =  
    (name, age) ->  
        System.out.println(  
            "Age: " + age  
        );  
  
BiConsumer<String, Integer> third =  
    (name, age) ->  
        System.out.println(  
            "Processing employee..."  
        );
```

Chain them:

```
BiConsumer<String, Integer> combined =  
    first  
        .andThen(second)  
        .andThen(third);
```

Call:

```
combined.accept("Anurag", 30);
```

Execution order:

first
↓
second
↓
third

Output:

Name: Anurag
Age: 30
Processing employee...

The order is guaranteed.

19. `andThen()` maintains the same inputs

This is a very important point.

Suppose:

```
BiConsumer<String, Integer> first =  
    (name, age) ->  
        System.out.println(name);
```

```
BiConsumer<String, Integer> second =  
    (name, age) ->  
        System.out.println(age);
```

Then:

```
BiConsumer<String, Integer> combined =  
    first.andThen(second);
```

Calling:

```
combined.accept("Anurag", 30);
```

passes:

"Anurag", 30

to **both** consumers.

Conceptually:

```
first.accept("Anurag", 30)  
    ↓  
second.accept("Anurag", 30)
```

There is no intermediate return value.

20. Exception behavior of andThen()

Suppose the first consumer throws an exception:

```
BiConsumer<String, Integer> first =  
    (name, age) -> {  
        throw new RuntimeException(  
            "First failed"  
        );  
    };
```

And:

```
BiConsumer<String, Integer> second =  
    (name, age) ->  
        System.out.println("Second");
```

Combine:

```
BiConsumer<String, Integer> combined =  
    first.andThen(second);
```

Calling:

```
combined.accept("Anurag", 30);
```

results in:

```
first.accept()  
↓  
exception  
↓  
second.accept() is NOT executed
```

So andThen() means **sequential execution**, not guaranteed execution of both operations regardless of failure.

21. Null handling in andThen()

The after consumer must not be null.

For example:

```
BiConsumer<String, Integer> after = null;
```

```
BiConsumer<String, Integer> combined =
```

```
first.andThen(after);
```

This results in:

NullPointerException

because Java requires a valid second BiConsumer.

22. BiConsumer with Map.forEach()

One of the most common practical uses of BiConsumer is with:

Map.forEach()

For example:

```
Map<String, Integer> marks =  
    new HashMap<>();
```

```
marks.put("Java", 90);  
marks.put("SQL", 85);  
marks.put("Python", 80);
```

Then:

```
marks.forEach(  
    (subject, mark) ->  
        System.out.println(  
            subject + " = " + mark  
        )  
);
```

The lambda has the shape:

(String, Integer) → void

which matches:

BiConsumer<String, Integer>

Output:

```
Java = 90  
SQL = 85  
Python = 80
```

23. Map.forEach() and BiConsumer relationship

The relevant Map method is conceptually:

```
void forEach(  
    BiConsumer<? super K, ? super V> action);
```

Suppose:

```
Map<String, Integer>
```

Then:

```
K = String  
V = Integer
```

So the action becomes:

```
(String, Integer) → void
```

which is exactly:

```
BiConsumer<String, Integer>
```

This is an excellent real-world example of why BiConsumer exists.

24. BiConsumer with a Map

You can explicitly declare it:

```
BiConsumer<String, Integer> printEntry =  
    (key, value) ->  
        System.out.println(  
            key + " = " + value  
        );
```

```
marks.forEach(printEntry);
```

This is equivalent to:

```
marks.forEach(  
    (key, value) ->  
        System.out.println(  
            key + " = " + value  
        )  
);
```

25. BiConsumer and method references

Suppose you have:

```
public static void print(  
    String name,  
    Integer age) {  
  
    System.out.println(  
        name + " : " + age  
    );  
}
```

You can use a method reference:

```
BiConsumer<String, Integer> printer =  
    BiConsumerDemo::print;
```

Then:

```
printer.accept("Anurag", 30);
```

Output:

```
Anurag : 30
```

The method reference matches:

`(String, Integer) → void`

which is exactly the BiConsumer signature.

26. Instance method reference

Suppose:

```
class EmployeePrinter {  
  
    public void print(  
        String name,  
        double salary) {  
  
        System.out.println(  
            name + " : " + salary  
        );  
    }  
}
```

```
    );  
  }  
}
```

Create:

```
EmployeePrinter printer =  
    new EmployeePrinter();
```

Then:

```
BiConsumer<String, Double> consumer =  
    printer::print;
```

Call:

```
consumer.accept(  
    "Amit",  
    50000.0  
);
```

Output:

Amit : 50000.0

27. Lambda vs method reference

Lambda:

```
BiConsumer<String, Integer> consumer =  
    (name, age) ->  
        print(name, age);
```

Method reference:

```
BiConsumer<String, Integer> consumer =  
    ClassName::print;
```

Both represent:

(String, Integer) → void

The method reference is simply a more concise way to refer to an existing compatible method.

28. Anonymous class equivalent

Lambda:

```
BiConsumer<String, Integer> print =  
    (name, age) ->  
        System.out.println(  
            name + " : " + age  
        );
```

Conceptually equivalent to:

```
BiConsumer<String, Integer> print =  
    new BiConsumer<String, Integer>() {  
  
        @Override  
        public void accept(  
            String name,  
            Integer age) {  
  
            System.out.println(  
                name + " : " + age  
            );  
        }  
    };
```

The lambda provides the implementation of:

accept()

29. BiConsumer with side effects

Consumer and BiConsumer are generally used when the operation is intended to have an **observable side effect**, such as:

- printing
- logging
- updating an object
- modifying a collection
- writing to a file
- sending a message
- updating a database
- invoking another operation

Example:

```
BiConsumer<String, Integer> log =  
    (name, age) ->
```

```
System.out.println(  
    "Employee: "  
    + name  
    + ", Age: "  
    + age  
);
```

The purpose isn't to calculate a return value.

The purpose is to **do something**.

30. BiConsumer and object mutation

For example:

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}
```

A BiConsumer can modify an employee:

```
BiConsumer<Employee, Double> increaseSalary =  
    (employee, amount) ->  
        employee.salary += amount;
```

Then:

```
Employee employee =  
    new Employee("Amit", 50000);  
  
increaseSalary.accept(employee, 5000.0);  
  
System.out.println(employee.salary);
```

Output:

55000.0

Notice:

(Employee, Double) → void

The employee object itself is modified.

31. BiConsumer for two-object operations

For example, transferring money conceptually:

```
BiConsumer<Account, Account> transfer =  
    (source, destination) -> {  
        // business logic  
    };
```

Its shape is:

(Account, Account) → void

The operation receives two accounts and performs an action.

Unlike:

BiFunction<Account, Account, Double>

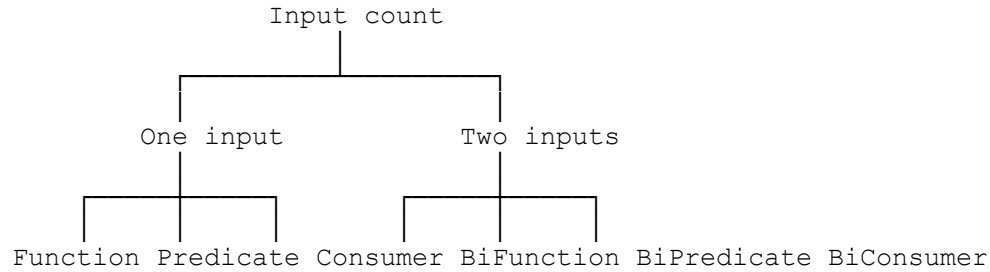
it doesn't need to return a value.

32. Important comparison of the functional interfaces

You can now see the major pattern:

Interface	Inputs	Output	Main method
Function<T,R>	1	R	apply()
BiFunction<T,U,R>	2	R	apply()
Predicate<T>	1	boolean	test()
BiPredicate<T,U>	2	boolean	test()
Consumer<T>	1	void	accept()
BiConsumer<T,U>	2	void	accept()
Supplier<T>	0	T	get()

This gives you a very useful pattern:



33. The three major Bi* interfaces

This is particularly useful for interviews:

BiFunction

$(T,U) \rightarrow R$

Calculate/transform and return a value.

```
BiFunction<Integer,Integer,Integer> add =  
    (a,b) -> a+b;
```

BiPredicate

$(T,U) \rightarrow \text{boolean}$

Test a condition.

```
BiPredicate<Integer,Integer> greater =  
    (a,b) -> a>b;
```

BiConsumer

$(T,U) \rightarrow \text{void}$

Perform an action.

```
BiConsumer<Integer,Integer> print =  
    (a,b) -> System.out.println(a+b);
```

Mnemonic:

BiFunction $\rightarrow$ RETURN
BiPredicate $\rightarrow$ TEST
BiConsumer $\rightarrow$ ACTION

34. BiConsumer method classification

This is the most important table for your question:

Method	Classification	Meaning
accept(T,U)	Abstract	Perform the two-argument action
andThen(...)	Default	Execute another action afterward

Therefore your original statement:

"BiConsumer functional interfaces include default methods such as accept(), andThen()."

should be corrected to:

BiConsumer<T,U> is a functional interface with one abstract method, accept(T,U), and one default method, andThen().

35. Complete example

```
import java.util.function.BiConsumer;

public class BiConsumerDemo {

    public static void main(String[] args) {

        BiConsumer<String, Integer> first =
            (name, age) ->
                System.out.println(
                    "Name = " + name
                );

        BiConsumer<String, Integer> second =
            (name, age) ->
                System.out.println(
                    "Age = " + age
                );

        // Execute first and then second
        BiConsumer<String, Integer> combined =
            first.andThen(second);

        combined.accept("Anurag", 30);
    }
}
```

```
}  
}
```

Output:

Name = Anurag
Age = 30

36. Complete Employee example

```
import java.util.function.BiConsumer;  
  
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}  
  
public class EmployeeBiConsumerDemo {  
  
    public static void main(String[] args) {  
  
        Employee e1 =  
            new Employee("Amit", 50000);  
  
        Employee e2 =  
            new Employee("Rahul", 70000);  
  
        BiConsumer<Employee, Employee> print =  
            (a, b) -> {  
  
                System.out.println(  
                    a.name + " : " + a.salary  
                );  
  
                System.out.println(  
                    b.name + " : " + b.salary  
                );  
            };  
  
        BiConsumer<Employee, Employee> total =  
            (a, b) ->  
                System.out.println(  
                    a.name + " : " + a.salary + " + " + b.name + " : " + b.salary + " = " + (a.salary + b.salary)  
                );  
    }  
}
```

```

        "Total Salary = "
        + (a.salary + b.salary)
    );

    BiConsumer<Employee, Employee> combined =
        print.andThen(total);

    combined.accept(e1, e2);
}
}

```

Output:

```

Amit : 50000.0
Rahul : 70000.0
Total Salary = 120000.0

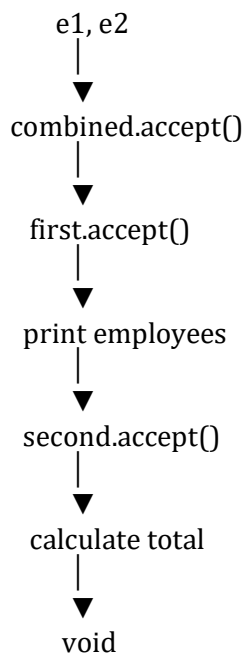
```

37. Internal execution sequence

For:

```
combined.accept(e1, e2);
```

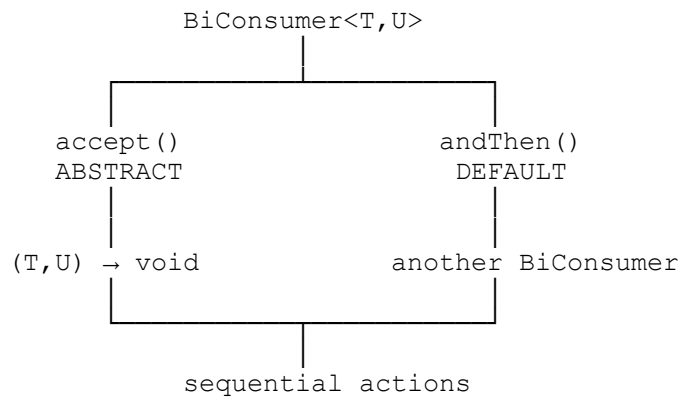
the algorithm is:



There is no returned intermediate value.

This is the key difference from `BiFunction.andThen()`.

38. Final mental model



The easiest mnemonic is:

`BiConsumer` = "Give me TWO values; I'll perform an ACTION."

For example:

```
BiConsumer<String, Integer> print =  
    (name, age) ->  
        System.out.println(  
            name + " : " + age  
        );
```

means:

```
String ┌───┐  
      │   │  
      └───┘ accept() ──> ACTION ──> void  
Integer ┌───┐
```

And:

```
first.andThen(second)
```

means:

same two inputs

```
┆  
┆  
▼  
first.accept()  
┆  
┆  
▼  
second.accept()  
┆
```

▼
void

Final classification

BiConsumer<T,U>
├── accept(T,U) → ABSTRACT
└── andThen() → DEFAULT

Key point: accept() is the single abstract/functional method. andThen() is the built-in default method that allows two BiConsumer actions to be executed sequentially.